JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 17 OF 17 - STUDY STEP DSA-17

Dynamic Programming: State, Transitions, and Optimization

Memoization, Tabulation, Reconstruction, Dimensions, and Space Compression

BY

Vinay Reddy Kalluri

Derive dynamic programs from state meaning, transitions, base cases, evaluation order, and reconstruction instead of memorizing tables.

STATE | TRANSITION | MEMOIZATION | TABULATION | RECONSTRUCTION | DP

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 16 - Greedy Algorithms`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume makes dynamic programming a specification process: define exactly what a state means, how it depends on smaller states, and what information can be compressed.

Readiness map

Decision	Evidence
START HERE	Subproblems overlap; A choice affects a reusable suffix or prefix state; Brute-force recursion repeats the same arguments; The problem asks for optimum, count, or feasibility under constraints.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-16; Recursion, arrays, strings, and complexity.
MOVE ON WHEN	Define DP state and transitions precisely; Choose memoization or tabulation; Reconstruct a solution and compress safe dimensions; Analyze state count times transition cost.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 16 - Greedy Algorithms	YOU ARE HERE **DSA 17 - Dynamic Programming**	SEGMENT COMPLETE

A note from Vinay

Dynamic programming starts when repeated decisions share the same future. Define the state in words, derive the transition from the last choice, and optimize memory only after the table is correct.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Dynamic Programming Foundations: Derive State Before Writing a Table

Dynamic programming is a way to evaluate repeated subproblems once. It is not a collection of unrelated formulas. A correct DP starts with a precise state sentence.

See repetition in a recursive tree

For Fibonacci:

TEXT

```
fib(5)
  fib(4)
    fib(3)
    fib(2)
  fib(3)       repeated
```

The recursion has optimal substructure - larger answers combine smaller answers - and overlapping subproblems - the same arguments recur.

JAVA

```java
static long fibonacciMemo(int n, long[] memo) {
    if (n <= 1) return n;
    if (memo[n] != -1L) return memo[n];
    memo[n] = fibonacciMemo(n - 1, memo) + fibonacciMemo(n - 2, memo);
    return memo[n];
}
```

Memoization preserves demand-driven recursion but still uses call-stack depth. Tabulation chooses an explicit evaluation order:

JAVA

```java
static long fibonacci(int n) {
    if (n < 0) throw new IllegalArgumentException("negative n");
    if (n <= 1) return n;
    long previous = 0L;
    long current = 1L;
    for (int index = 2; index <= n; index++) {
        long next = Math.addExact(previous, current);
        previous = current;
        current = next;
    }
    return current;
}
```

Space compression is safe because each state needs only the previous two states. Numeric overflow remains a separate correctness boundary.

The six questions for every DP

1. **State:** What does `dp[...]` mean in one complete sentence?
2. **Choices:** What can the current decision do?
3. **Transition:** Which smaller states produce this state?
4. **Base:** Which states are known without recurrence?
5. **Order:** Are dependencies computed before use?
6. **Answer:** Which state or aggregate is returned?

Add two more for SDE-2:

7. **Reconstruction:** What parent/choice evidence recovers an actual solution?
8. **Compression:** Which older states are provably dead?

Example: House Robber from a state sentence

State:

```text
dp[i] = maximum amount obtainable from houses in indexes [0, i)
```

At boundary `i`, either skip house `i-1` and keep `dp[i-1]`, or take it and add its value to `dp[i-2]`.

```text
dp[i] = max(dp[i-1], dp[i-2] + value[i-1])
dp[0] = 0
dp[1] = max(0, value[0]) under an allow-skip-all contract
```

This formulation exposes boundary indexes and handles empty input naturally.

Memoization versus tabulation

Question	Memoization	Tabulation
Evaluation	only reached states	chosen order, often all states
Stack	recursive depth	usually none
Sparse state	can avoid unreachable states	may need explicit sparse map
Ordering	follows recursion	must be derived
Debugging	mirrors recurrence	table can be inspected

Neither is automatically faster. Compare reachable-state count, call overhead, iteration locality, and stack safety.

One-dimensional and two-dimensional state

Add a dimension only when the future answer depends on that information. For 0/1 knapsack:

TEXT

```
dp[item][capacity] = best value using first item items within capacity
```

The item dimension distinguishes which choices remain. The capacity dimension distinguishes remaining resource. The transition compares skipping and taking the current item.

Evaluation order is correctness

When compressing 0/1 knapsack to one dimension, capacities must iterate downward. Upward iteration would reuse the current item multiple times, silently changing the problem to unbounded knapsack.

JAVA

```
for each item:
    for capacity from limit down to itemWeight:
        dp[capacity] = max(dp[capacity], dp[capacity - itemWeight] + itemValue)
```

For unlimited reuse, upward iteration is often intentional. Loop direction encodes the reuse contract.

Counting, feasibility, and optimization differ

The same input shape can ask:

- feasibility: can a sum be formed? use boolean state;
- count: how many ways? use additive state and define order/combinations;
- minimum: fewest items? use infinity sentinel and min transition;
- maximum: best value? use max transition;
- reconstruction: which choices? store parents or retain enough table state.

Do not transplant a recurrence without restating the state.

Complexity from state space

TEXT

```
time = number of reachable states * transition work per state
space = stored states + recursion depth + reconstruction/output
```

For `n * capacity` knapsack states with O(1) transition work, time is O(n * capacity). This is pseudo-polynomial because capacity is a numeric value, not its bit-length.

Compression caution

Compression can destroy information needed to reconstruct choices, process dependencies in the wrong order, or obscure correctness. Derive a full correct table first, draw dependency arrows, then retain only live layers.

TRY IT | Foundation checkpoint

1. State House Robber DP in one sentence.
2. Why does memoization reduce repeated work but not recursion depth?
3. Why does 0/1 knapsack iterate capacity downward after compression?
4. What distinguishes combination count from permutation count?
5. How do you derive complexity without memorizing it?

SDE-2 CORE CHAPTER

Chapter 2 - SDE-2 Dynamic Programming: Derivation, Reconstruction, and Optimization

Why dynamic programming feels harder than it is

Dynamic programming (DP) is not a list of formulas. It is a method for organizing repeated subproblems when an optimum or count can be composed from smaller states. Most failures come from choosing an ambiguous state, omitting a base case, evaluating dependencies in the wrong order, or compressing storage before correctness is clear.

At SDE-2 level, derive the recurrence out loud, count states and transitions, identify numeric and memory limits, and reconstruct the requested choices rather than returning only a score. This chapter builds a repeatable derivation across one-dimensional decisions, grids, knapsack, subset and coin problems, subsequences, edit operations, stock state machines, and advanced interval/tree/bitmask families.

Learning objectives

After completing this chapter, you should be able to:

- distinguish overlapping subproblems from greedy-choice structure and brute force;
- derive state, transition, base cases, evaluation order, and answer location;
- move between memoization and tabulation;
- count complexity as number of reachable states times work per state;
- solve house robber, grid path, 0/1 knapsack, subset sum, and coin change;
- derive LIS, LCS, edit distance, and stock transaction states;
- reconstruct selected items or a sequence from a DP table;
- compress memory only when discarded states are no longer dependencies;
- recognize interval, tree, and bitmask DP boundaries; and
- discuss overflow, unreachable sentinels, recursion depth, allocation, and production scale.

The six-step derivation protocol

For every DP problem, write these six lines before code.

1. **State:** what exact subproblem does `dp[...]` answer?
2. **Choice/transition:** which first or last decision relates this state to smaller states?
3. **Base:** what are the smallest valid states and impossible states?
4. **Order:** in which order are all dependencies already known?
5. **Answer:** which state or aggregate is the requested result?
6. **Complexity:** how many states exist and how much work evaluates each?

Example for climbing `n` steps using moves of one or two:

TEXT

```
state: ways(i) = number of ways to reach exactly step i
transition: ways(i) = ways(i - 1) + ways(i - 2)
base: ways(0)=1, ways(negative)=0
order: increasing i, or recursive memoization
answer: ways(n)
states: n + 1, O(1) work each -> O(n) time
```

The empty construction base `ways(0)=1` is important: it provides one way to complete a recurrence exactly, not zero physical movements.

Recognition: DP versus alternatives

DP is promising when:

- a recursive search revisits the same parameter combination;
- the answer for a prefix, suffix, interval, capacity, or finite state can be composed from smaller ones;
- choices interact, so a local greedy ranking lacks an exchange proof;
- the input asks for an optimum, count, feasibility, or reconstruction; and
- the state space is small enough to enumerate.

DP is not automatically appropriate when states are unique, an invariant yields a linear greedy solution, or dimensions make the state space exponential. A memoized exponential state definition is still exponential.

Memoization versus tabulation

Top-down memoization

Write the recurrence as a recursive function and cache results by state. Benefits:

- follows the mathematical definition;
- computes only reachable states; and
- can be easier for irregular transitions.

Costs:

- recursion frames and possible `StackOverflowError`;
- hash/boxing overhead for non-array keys; and
- less predictable locality.

The memo must distinguish "uncomputed" from a legitimate result such as zero. Use a separate seen array, nullable boxed entries, or a sentinel outside the result domain.

Bottom-up tabulation

Allocate states and fill them in dependency order. Benefits:

- no recursion depth risk;
- predictable memory and often better locality; and
- easier rolling-array compression.

Costs:

- may compute unreachable states; and
- order can be less intuitive for interval or tree structures.

The toolkit implements climbing both ways. Both take `O(n)` time. The tabulated version keeps only two predecessors and uses `O(1)` space; memoization uses `O(n)` memo and stack.

Pattern 1: house robber / nonadjacent selection

Given values along a line, select a maximum-sum subset with no adjacent indexes. Define:

```text
dp[i] = best value using prefix [0, i)
```

At last prefix item `i - 1`, either skip it and keep `dp[i - 1]`, or take it and add its value to `dp[i - 2]`:

```text
dp[i] = max(dp[i - 1], dp[i - 2] + value[i - 1])
dp[0] = 0
dp[1] = max(0, value[0])
```

Invariant while scanning: `previousOne` and `previousTwo` are optimal results for the two preceding prefix lengths. Only those states are dependencies, so storage compresses to `O(1)`.

For `[2,7,9,3,1]`, prefix optima are `0,2,7,11,11,12`; choose indexes 0,2,4 for 12. If negative values are allowed and selecting nothing is legal, zero is a valid answer. If at least one item is required, bases and result change.

Time `O(n)`, space `O(1)` for score only. Reconstructing indexes normally needs the full table or stored decisions.

Pattern 2: grid path DP

For minimum cost from top-left to bottom-right with moves only right and down:

```text
dp[row][col] = grid[row][col] + min(dp[row-1][col], dp[row][col-1])
```

The graph of legal moves is a DAG ordered by `row + col`. First cell is its own cost; first row can come only from the left; first column only from above.

For grid:

```text
1 3 1
1 5 1
4 2 1
```

minimum prefix rows become `[1,4,5]`, `[2,7,6]`, `[6,8,7]`, answer 7. One path is right, right, down, down.

Time is `O(rows * cols)`. Score-only storage compresses to one row of `O(cols)`: before updating `dp[col]` it means the value from above; after update it means current row, while `dp[col-1]` is left. Obstacles need an unreachable sentinel and guarded addition. Arbitrary four-direction movement creates cycles and calls for graph shortest-path algorithms, not this acyclic recurrence.

Pattern 3: 0/1 knapsack

Each item has positive weight and nonnegative value and may be taken at most once. With capacity `c`:

```text
dp[i][c] = best value using first i items with capacity c
```

Skip item `i-1`, or take it if its weight fits:

```text
dp[i][c] = max(dp[i-1][c],
               value[i-1] + dp[i-1][c-weight[i-1]])
```

Both branches depend on the previous item row, enforcing at-most-once use. Base row zero is all zero.

TRACE IT | Dry run and reconstruction

Weights `[2,3,4]`, values `[4,5,7]`, capacity 6. Best value is 11 from items 0 and 2. Starting at `dp[3][6]`, compare with `dp[2][6]`; the value differs, so item 2 was selected and capacity becomes 2. Compare `dp[2][2]` with `dp[1][2]`; they match, so skip item 1. Item 0 then differs from the zero row, so select it.

State count is `(n+1)(capacity+1)`, each `O(1)`, yielding `O(nC)` time and space. This is pseudo-polynomial: capacity is numeric magnitude, not input bit length.

The runnable full-table implementation rejects a table above an explicit cell budget and computes dimensions in `long` before any `+1` result reaches an array allocation. In particular, `capacity == Integer.MAX_VALUE` cannot become a negative column count through overflow. A production API should choose its budget from measured heap limits or use score-only compression when reconstruction is unnecessary.

For score only, compress to `O(C)` and iterate capacities downward. Downward order ensures `dp[c-weight]` still represents the previous item row. Iterating upward accidentally permits the same item repeatedly and solves an unbounded variant. Reconstruction becomes harder after compression; retain decisions, rerun portions, or accept full storage.

Pattern 4: subset sum

For nonnegative values, `possible[s]` says whether some processed subset totals `s`. Seed `possible[0]=true`. For each value, iterate `s` downward from target and set:

```text
possible[s] |= possible[s - value]
```

Invariant after an item: `possible[s]` represents subsets using only processed items, each at most once. Descending order prevents reading a state just created by the same item.

Time `O(n * target)`, space `O(target)`. Zero values do not change feasibility but matter for counting subsets. Negative values invalidate the simple `0..target` index range; use an offset range, a set of reachable sums, meet-in-the-middle, or another constraint-specific method.

Equal-partition is subset sum with target `total/2` after checking the total is even. Widen total before addition.

Pattern 5: coin change distinctions

"Coin change" names several different DPs.

Minimum number of coins, unlimited reuse

```text
dp[amount] = minimum coins to form amount
dp[0] = 0
dp[a] = 1 + min(dp[a - coin]) over fitting coins
```

Use an unreachable sentinel and never add one to it blindly. Loop amount increasing so smaller amounts are known. Time `O(target * coinTypes)`, space `O(target)`.

For coins `[1,3,4]`, target 6, minimum is two coins `3+3`; the locally largest coin 4 leaves 2 and produces `4+1+1`, showing why a generic largest-coin greedy rule fails.

Count combinations, unlimited reuse

Seed `ways[0]=1`; loop coins outside and amounts increasing inside. This counts each multiset once. Swapping loop order counts ordered sequences/permutations instead. For 0/1 use, amounts descend. Loop order is part of the problem semantics, not a micro-optimization.

Counts can grow far beyond `long`; define modulus or use `BigInteger` when exact large counts matter.

Pattern 6: longest increasing subsequence

Quadratic DP defines `dp[i]` as LIS length ending exactly at `i`:

```text
dp[i] = 1 + max(dp[j]) for j < i and values[j] < values[i]
```

This takes `O(n^2)` time and `O(n)` space and reconstructs naturally with predecessor indexes.

The `O(n log n)` method maintains `tails[length-1]`, the smallest possible tail value of any increasing subsequence of that length seen so far. For each value, binary-search the first tail greater than or equal to it and replace that tail, or extend the array.

Invariant: tails are sorted, and a smaller tail is at least as extendable as a larger tail for the same length. Replacing a tail does not claim the tails array itself is a subsequence; it preserves existence of some subsequence for each length.

For `[10,9,2,5,3,7,101,18]`, tails evolve `[10]`, `[9]`, `[2]`, `[2,5]`, `[2,3]`, `[2,3,7]`, `[2,3,7,101]`, `[2,3,7,18]`; length 4. Strict LIS uses lower bound `>=`; nondecreasing subsequence uses upper bound `>`.

Reconstructing the `O(n log n)` sequence requires predecessor and tail-index arrays, not just tail values.

Pattern 7: longest common subsequence

For code-point arrays `a` and `b`:

```
TEXT
dp[i][j] = LCS length of prefixes a[0..i) and b[0..j)
```

If last code points match, include them: `1 + dp[i-1][j-1]`. Otherwise skip one side: `max(dp[i-1][j], dp[i][j-1])`. Empty prefix rows/columns are zero.

For `ABCBDAB` and `BDCABA`, an LCS length is 4, such as `BCBA`. Reconstruction walks backward: matching symbols are chosen; otherwise move toward a neighboring state with the same optimal value. Ties permit multiple correct sequences, so deterministic tie policy matters.

Time and full reconstruction space are `O(nm)`. Score only compresses to `O(min(n,m))`, but ordinary backtracking information is lost. Hirschberg's algorithm reconstructs with linear additional space through divide and conquer when needed.

Pattern 8: edit distance

Levenshtein distance permits insertion, deletion, and substitution at unit cost.

```
TEXT
dp[i][j] = minimum edits from first i code points to first j
base: dp[i][0]=i, dp[0][j]=j
if equal: dp[i][j]=dp[i-1][j-1]
else: 1 + min(delete dp[i-1][j],
              insert dp[i][j-1],
              replace dp[i-1][j-1])
```

Every transition identifies the last operation, so optimal substructure follows by removing that operation. State count `O(nm)`, transition work constant. The toolkit compresses to two rows. To reconstruct the edit script, retain the table or decisions.

Costs can be weighted, transposition can be added under a different recurrence, and Unicode normalization/unit choices belong to the contract. Code-point distance is not grapheme or linguistic distance.

Pattern 9: stock trading as a state machine

For at most `k` transactions, define:

- `cash[t]`: best profit after at most `t` completed sales while holding no stock;
- `hold[t]`: best profit after buying for transaction `t` and currently holding one stock.

For each price:

```text
hold[t] = max(hold[t], cash[t-1] - price)
cash[t] = max(cash[t], hold[t] + price)
```

Impossible hold states start at negative infinity. The implementation uses a safe sentinel so addition cannot overflow. State count is `O(k)` per day; time `O(nk)`, space `O(k)`.

When `k >= n/2`, the transaction cap cannot bind because each profitable transaction needs at least a buy and later sell day. Sum every positive adjacent increase in `O(n)` time.

Cooldown, fees, multiple simultaneous holdings, and mandatory transactions change state and transitions. Write the state meaning first instead of patching conditionals onto a remembered formula.

Advanced pattern boundaries

Interval DP

State spans a contiguous interval, often `[left,right)`. Choose a split or last action inside it. Matrix-chain multiplication, optimal parenthesization, and burst balloons are examples. Fill shorter intervals before longer ones. Typical state count is `O(n^2)` and trying every split yields `O(n^3)` time.

Tree DP

Root the tree and compute states after children. House robber on a tree returns two values per node: best if this node is excluded and best if included. Included forces children excluded; excluded may choose each child's better state. Complexity is linear, but recursion depth may require an iterative postorder.

Bitmask DP

For small `n`, a mask represents a chosen subset. Traveling-salesperson-style state `dp[mask][last]` stores best cost to visit exactly `mask` and end at `last`. There are `n * 2^n` states and up to `n` transitions each, `O(n^2 2^n)` time. This is powerful around `n` in the teens or low twenties, not a polynomial escape from large input.

Digit DP

Count numbers satisfying a property with state such as `(position, tight, started, summary)`. Memoization is valid only when the state includes every future-relevant constraint. This is a recognition boundary rather than a default interview tool.

Reconstruction and compression tension

A score is not always the requested answer. To reconstruct:

- store a parent/choice per state;
- compare a full state with the transition that could have produced it;
- retain predecessor indexes; or
- rerun selected subproblems when memory is more valuable than time.

Compression is safe only when future values no longer need discarded states. Iteration direction matters in knapsack. In-place grid rows rely on old-above/new-left meanings. Reconstruction may require the information compression removes. State this trade-off before claiming `O(1)` space for a problem that must return a path or chosen set.

Runnable Java 21 reference implementation

Run with `java -ea DynamicProgrammingPatterns`.

JAVA (continued 1/11)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public final class DynamicProgrammingPatterns {
    private static final long NEGATIVE_INFINITY = Long.MIN_VALUE / 4;
    private static final long MAX_KNAPSACK_CELLS = 20_000_000L;

    private DynamicProgrammingPatterns() {
    }

    public record KnapsackResult(long value, List<Integer> selectedIndexes) {
    }

    public static long climbWaysMemoized(int steps) {
        validateStepCount(steps);
        long[] memo = new long[steps + 1];
        Arrays.fill(memo, -1);
        return climb(steps, memo);
    }

    public static long climbWaysTabulated(int steps) {
        validateStepCount(steps);
        if (steps == 0) {
            return 1;
        }
        long twoBack = 1;
        long oneBack = 1;
        for (int step = 2; step <= steps; step++) {
            long current = Math.addExact(oneBack, twoBack);
            twoBack = oneBack;
            oneBack = current;
        }
```

JAVA (continued 2/11)

```
        return oneBack;
    }

    public static long maxNonAdjacentSum(int[] values) {
        requireArray(values);
        long twoBack = 0;
        long oneBack = 0;
        for (int value : values) {
            long current = Math.max(oneBack, Math.addExact(twoBack, value));
            twoBack = oneBack;
            oneBack = current;
        }
        return oneBack;
    }

    public static long minimumGridPath(int[][] grid) {
        int cols = validateGrid(grid);
        long[] best = new long[cols];
        for (int row = 0; row < grid.length; row++) {
            for (int col = 0; col < cols; col++) {
                if (row == 0 && col == 0) {
                    best[col] = grid[row][col];
                } else if (row == 0) {
                    best[col] = Math.addExact(best[col - 1], grid[row][col]);
                } else if (col == 0) {
                    best[col] = Math.addExact(best[col], grid[row][col]);
                } else {
                    best[col] = Math.addExact(Math.min(best[col], best[col - 1]),
                            grid[row][col]);
                }
            }
        }
        return best[cols - 1];
    }
```

JAVA (continued 3/11)

```
    public static KnapsackResult knapsack01(int[] weights, int[] values, int capacity) {
        if (weights == null || values == null || weights.length != values.length
                || capacity < 0) {
            throw new IllegalArgumentException("invalid knapsack input");
        }
        int n = weights.length;
        long rows = (long) n + 1;
        long columns = (long) capacity + 1;
        long cells = Math.multiplyExact(rows, columns);
        if (rows > Integer.MAX_VALUE || columns > Integer.MAX_VALUE
                || cells > MAX_KNAPSACK_CELLS) {
            throw new IllegalArgumentException("knapsack table exceeds allocation budget");
        }
        long[][] best = new long[n + 1][capacity + 1];
        for (int i = 1; i <= n; i++) {
            if (weights[i - 1] <= 0 || values[i - 1] < 0) {
                throw new IllegalArgumentException("positive weights and nonnegative values required");
            }
            for (int remaining = 0; remaining <= capacity; remaining++) {
                best[i][remaining] = best[i - 1][remaining];
                if (weights[i - 1] <= remaining) {
                    best[i][remaining] = Math.max(best[i][remaining],
                            Math.addExact(values[i - 1],
                                    best[i - 1][remaining - weights[i - 1]]));
                }
            }
        }
        List<Integer> selected = new ArrayList<>();
        int remaining = capacity;
        for (int i = n; i >= 1; i--) {
            if (best[i][remaining] != best[i - 1][remaining]) {
                selected.add(i - 1);
                remaining -= weights[i - 1];
```

JAVA (continued 4/11)

```
            }
        }
        Collections.reverse(selected);
        return new KnapsackResult(best[n][capacity], List.copyOf(selected));
    }

    public static boolean subsetSum(int[] values, int target) {
        requireArray(values);
        if (target < 0 || target == Integer.MAX_VALUE) {
            throw new IllegalArgumentException("target must be in 0..Integer.MAX_VALUE-1");
        }
        boolean[] possible = new boolean[target + 1];
        possible[0] = true;
        for (int value : values) {
            if (value < 0) {
                throw new IllegalArgumentException("values must be nonnegative");
            }
            for (int sum = target; sum >= value; sum--) {
                possible[sum] |= possible[sum - value];
            }
        }
        return possible[target];
    }

    public static int minimumCoins(int[] coins, int target) {
        requireArray(coins);
        if (target < 0 || target == Integer.MAX_VALUE) {
            throw new IllegalArgumentException("invalid target");
        }
        for (int coin : coins) {
            if (coin <= 0) {
                throw new IllegalArgumentException("coins must be positive");
            }
```

JAVA (continued 5/11)

```
        }
        int unreachable = target + 1;
        int[] best = new int[target + 1];
        Arrays.fill(best, unreachable);
        best[0] = 0;
        for (int amount = 1; amount <= target; amount++) {
            for (int coin : coins) {
                if (coin <= amount && best[amount - coin] != unreachable) {
                    best[amount] = Math.min(best[amount], best[amount - coin] + 1);
                }
            }
        }
        return best[target] == unreachable ? -1 : best[target];
    }

    public static int lisLength(int[] values) {
        requireArray(values);
        int[] tails = new int[values.length];
        int size = 0;
        for (int value : values) {
            int low = 0;
            int high = size;
            while (low < high) {
                int middle = low + (high - low) / 2;
                if (tails[middle] < value) {
                    low = middle + 1;
                } else {
                    high = middle;
                }
            }
            tails[low] = value;
            if (low == size) {
                size++;
```

JAVA (continued 6/11)

```
            }
        }
        return size;
    }

    public static String longestCommonSubsequence(String first, String second) {
        requireString(first);
        requireString(second);
        int[] a = first.codePoints().toArray();
        int[] b = second.codePoints().toArray();
        int[][] length = new int[a.length + 1][b.length + 1];
        for (int i = 1; i <= a.length; i++) {
            for (int j = 1; j <= b.length; j++) {
                if (a[i - 1] == b[j - 1]) {
                    length[i][j] = length[i - 1][j - 1] + 1;
                } else {
                    length[i][j] = Math.max(length[i - 1][j], length[i][j - 1]);
                }
            }
        }
        List<Integer> reversed = new ArrayList<>();
        int i = a.length;
        int j = b.length;
        while (i > 0 && j > 0) {
            if (a[i - 1] == b[j - 1]) {
                reversed.add(a[i - 1]);
                i--;
                j--;
            } else if (length[i - 1][j] >= length[i][j - 1]) {
                i--;
            } else {
                j--;
            }
```

JAVA (continued 7/11)

```
        }
        Collections.reverse(reversed);
        StringBuilder result = new StringBuilder();
        for (int point : reversed) {
            result.appendCodePoint(point);
        }
        return result.toString();
    }

    public static int editDistance(String first, String second) {
        requireString(first);
        requireString(second);
        int[] a = first.codePoints().toArray();
        int[] b = second.codePoints().toArray();
        if (b.length > a.length) {
            int[] temporary = a;
            a = b;
            b = temporary;
        }
        int[] previous = new int[b.length + 1];
        int[] current = new int[b.length + 1];
        for (int j = 0; j <= b.length; j++) {
            previous[j] = j;
        }
        for (int i = 1; i <= a.length; i++) {
            current[0] = i;
            for (int j = 1; j <= b.length; j++) {
                if (a[i - 1] == b[j - 1]) {
                    current[j] = previous[j - 1];
                } else {
                    current[j] = 1 + Math.min(previous[j - 1],
                            Math.min(previous[j], current[j - 1]));
                }
```

JAVA (continued 8/11)

```
            }
            int[] temporary = previous;
            previous = current;
            current = temporary;
        }
        return previous[b.length];
    }

    public static long maxStockProfit(int[] prices, int transactions) {
        requireArray(prices);
        if (transactions < 0) {
            throw new IllegalArgumentException("transactions must be nonnegative");
        }
        for (int price : prices) {
            if (price < 0) {
                throw new IllegalArgumentException("prices must be nonnegative");
            }
        }
        if (transactions == 0 || prices.length < 2) {
            return 0;
        }
        if (transactions >= prices.length / 2) {
            long profit = 0;
            for (int day = 1; day < prices.length; day++) {
                if (prices[day] > prices[day - 1]) {
                    profit += (long) prices[day] - prices[day - 1];
                }
            }
            return profit;
        }
        long[] cash = new long[transactions + 1];
        long[] hold = new long[transactions + 1];
        Arrays.fill(hold, NEGATIVE_INFINITY);
```

JAVA (continued 9/11)

```
        for (int price : prices) {
            for (int completed = 1; completed <= transactions; completed++) {
                hold[completed] = Math.max(hold[completed], cash[completed - 1] - price);
                cash[completed] = Math.max(cash[completed], hold[completed] + price);
            }
        }
        return cash[transactions];
    }

    private static long climb(int steps, long[] memo) {
        if (steps <= 1) {
            return 1;
        }
        if (memo[steps] != -1) {
            return memo[steps];
        }
        memo[steps] = Math.addExact(climb(steps - 1, memo), climb(steps - 2, memo));
        return memo[steps];
    }

    private static void validateStepCount(int steps) {
        if (steps < 0 || steps > 91) {
            throw new IllegalArgumentException("steps must be in 0..91 for long output");
        }
    }

    private static int validateGrid(int[][] grid) {
        if (grid == null || grid.length == 0 || grid[0] == null
                || grid[0].length == 0) {
            throw new IllegalArgumentException("nonempty grid required");
        }
        int cols = grid[0].length;
        for (int[] row : grid) {
```

JAVA (continued 10/11)

```
            if (row == null || row.length != cols) {
                throw new IllegalArgumentException("grid must be rectangular");
            }
        }
        return cols;
    }

    private static void requireArray(int[] values) {
        if (values == null) {
            throw new IllegalArgumentException("array must not be null");
        }
    }

    private static void requireString(String value) {
        if (value == null) {
            throw new IllegalArgumentException("string must not be null");
        }
    }

    public static void main(String[] args) {
        assert climbWaysMemoized(10) == 89;
        assert climbWaysTabulated(10) == 89;
        assert maxNonAdjacentSum(new int[] {2, 7, 9, 3, 1}) == 12;
        assert maxNonAdjacentSum(new int[] {-4, -2}) == 0;
        assert minimumGridPath(new int[][] {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}}) == 7;

        KnapsackResult packed = knapsack01(new int[] {2, 3, 4},
                new int[] {4, 5, 7}, 6);
        assert packed.value() == 11;
        assert packed.selectedIndexes().equals(List.of(0, 2));
        boolean hugeCapacityRejected = false;
        try {
            knapsack01(new int[0], new int[0], Integer.MAX_VALUE);
```

JAVA (continued 11/11)

```
        } catch (IllegalArgumentException expected) {
            hugeCapacityRejected = true;
        }
        assert hugeCapacityRejected;
        assert subsetSum(new int[] {3, 34, 4, 12, 5, 2}, 9);
        assert !subsetSum(new int[] {3, 34, 4, 12, 5, 2}, 30);
        assert minimumCoins(new int[] {1, 3, 4}, 6) == 2;
        boolean invalidCoinRejected = false;
        try {
            minimumCoins(new int[] {0}, 0);
        } catch (IllegalArgumentException expected) {
            invalidCoinRejected = true;
        }
        assert invalidCoinRejected;

        assert lisLength(new int[] {10, 9, 2, 5, 3, 7, 101, 18}) == 4;
        String lcs = longestCommonSubsequence("ABCBDAB", "BDCABA");
        assert lcs.length() == 4;
        assert isSubsequence(lcs, "ABCBDAB") && isSubsequence(lcs, "BDCABA");
        assert editDistance("kitten", "sitting") == 3;
        assert maxStockProfit(new int[] {3, 2, 6, 5, 0, 3}, 2) == 7;
    }

    private static boolean isSubsequence(String candidate, String source) {
        int index = 0;
        for (int point : source.codePoints().toArray()) {
            if (index < candidate.length() && candidate.codePointAt(index) == point) {
                index += Character.charCount(point);
            }
        }
        return index == candidate.length();
    }
}
```

Complexity and state table

Family	States	Work/state	Time	Compressed space
climb/house robber	`O(n)`	`O(1)`	`O(n)`	`O(1)`
grid right/down	`O(rows*cols)`	`O(1)`	`O(rows*cols)`	`O(cols)`
0/1 knapsack	`O(nC)`	`O(1)`	`O(nC)`	`O(C)` score only
subset sum	`O(nT)` conceptual	`O(1)`	`O(nT)`	`O(T)`
minimum coin change	`O(T)`	`O(coins)`	`O(T*coins)`	`O(T)`
LIS quadratic	`O(n)` states	`O(n)`	`O(n^2)`	`O(n)`
LIS tails	`O(n)`	`O(log n)` search	`O(n log n)`	`O(n)`
LCS/edit distance	`O(nm)`	`O(1)`	`O(nm)`	`O(min(n,m))` score
stock at most k	`O(nk)`	`O(1)`	`O(nk)`	`O(k)`
interval split DP	`O(n^2)`	often `O(n)`	often `O(n^3)`	problem-specific
bitmask last-state	`O(n2^n)`	up to `O(n)`	`O(n^2 2^n)`	exponential

WATCH OUT | Edge cases and common mistakes

1. **State lacks future information.** If two histories with the same key permit different futures, the state is incomplete.
2. **Legitimate zero used as uncomputed.** Separate memo status from values.
3. **Base for empty construction wrong.** Counts often use one empty way; maxima may use zero or negative infinity.
4. **Impossible state added blindly.** Guard sentinels before arithmetic.
5. **Iteration direction changes item reuse.** Descending capacity is 0/1; ascending enables unbounded reuse.
6. **Combination and sequence counts confused.** Coin/amount loop order changes meaning.
7. **All-negative selection contract.** Decide whether choosing nothing is legal.
8. **Grid recurrence used on cyclic moves.** Use graph algorithms when dependencies are not acyclic.
9. **LIS strictness wrong.** Lower-bound versus upper-bound determines duplicate handling.
10. **Compressed score claimed to reconstruct choices.** Preserve decisions or explain recomputation.
11. **Pseudo-polynomial called polynomial in input length.** Capacity/target magnitude matters.
12. **Count overflow.** Combinatorial counts exceed `long` quickly.
13. **Recursive memo stack ignored.** State count does not bound call depth safely for Java.
14. **Huge table allocated from untrusted input.** Validate product before allocation.
15. **Text unit unspecified.** Code points, UTF-16 units, and graphemes produce different sequence states.

SDE-2 production follow-ups

- **Allocation guards:** compute table dimensions in `long`, enforce budgets, and fail before allocating. `O(nm)` may be impossible even when time sounds acceptable.
- **Sparse states:** memo maps can save memory when few states are reachable, but hashing and boxing add cost and cardinality still needs a bound.
- **Rolling storage:** reuse primitive arrays for locality and less GC pressure; clear only required ranges.
- **Overflow contract:** use exact arithmetic, modular counts, saturation, `BigInteger`, or explicit rejection. Silent wraparound destroys optimal comparisons.
- **Reconstruction audit:** return named choices and validate their score independently. This catches tie/backtracking bugs.
- **Incremental changes:** DP tables generally become stale when inputs change. Determine whether localized recomputation is valid or rebuild from a versioned snapshot.
- **Parallelism:** wavefront/grid phases or independent states may parallelize, but dependencies, synchronization, and memory bandwidth can dominate.
- **Approximation:** capacity or state explosion may require scaling, pruning, beam search, or approximation schemes. Label loss of exactness.
- **Persistence:** checkpointing large tables can cost more than recomputation; measure serialization and version recurrence semantics.
- **Observability:** record state count, reachable-state ratio, table bytes, cache hit rate, and reconstruction validation, not raw sensitive inputs.

TRY IT | Exercises with model checkpoints

Exercise 1: reconstruct house robber indexes

Return one optimal nonadjacent index set.

Model checkpoints: retain full prefix scores or decisions; backtrack by comparing skip with take; define ties deterministically; verify no adjacent indexes and recompute sum.

Exercise 2: grid with obstacles and path

Return minimum cost and coordinates.

Model checkpoints: unreachable sentinel; never add to sentinel; store parent direction or compare table predecessors; start/end blocked behavior; `O(rows*cols)` output-aware memory.

Exercise 3: count coin combinations

Count ways to form target with unlimited coins.

Model checkpoints: validate distinct positive denominations or define duplicate treatment; coins outer, amount ascending; `ways[0]=1`; choose `BigInteger` or modulus; contrast permutation loop order.

Exercise 4: edit script

Return insert/delete/replace operations, not only distance.

Model checkpoints: full table or divide-and-conquer reconstruction; define tie order; indexes shift as edits apply, so represent operations against prefixes or apply from a safe direction; verify script transforms input.

Exercise 5: LIS reconstruction

Extend the `O(n log n)` method to return indexes.

Model checkpoints: `tailIndex[length]` stores input index; predecessor of current is previous length's tail index; final chain backtracks; tails values alone are not a sequence; strict duplicate policy.

Exercise 6: tree independent set

Maximum-weight set of nonadjacent tree vertices.

Model checkpoints: per node return `(excluded,included)`; included adds excluded children, excluded adds max child states; postorder; parent prevents traversal back edge; iterative plan for deep trees.

Exercise 7: traveling salesperson bitmask DP

Find minimum Hamiltonian tour for small `n`.

Model checkpoints: state `(mask,last)` includes start; transition from previous endpoint; unreachable sentinel; close tour to start; `O(n^2 2^n)` time and `O(n2^n)` space; reject large n before shifting/allocating.

Interview answer checklist

- [ ] I wrote a one-sentence state meaning with exact indexes/ranges.
- [ ] I derived every transition from a last or first choice.
- [ ] I covered empty, impossible, and smallest bases.
- [ ] My evaluation order satisfies every dependency.
- [ ] I identified the exact answer state.
- [ ] I counted states times transition work.
- [ ] I distinguished pseudo-polynomial and exponential complexity.
- [ ] I compressed only after proving which history is unnecessary.
- [ ] I preserved enough information for requested reconstruction.
- [ ] I defined overflow, text units, and allocation limits.
- [ ] I can compare memoization, tabulation, greedy, graph, and search alternatives.

RECAP | Summary

Dynamic programming becomes systematic when every solution follows state, transition, base, order, answer, and complexity. Memoization mirrors recursion; tabulation exposes dependency order and compression. House robber retains two prefix states; grid paths follow a DAG; 0/1 knapsack and subset sum depend on descending capacity; coin loop order encodes reuse and ordering semantics. LIS has quadratic and tails-based formulations. LCS and edit distance align two sequences, while stock trading is a finite state machine over days and transaction counts. Interval, tree, and bitmask DP extend the same method. The SDE-2 distinction is honest state counting, reconstructable outputs, safe sentinels and arithmetic, and memory-aware production boundaries.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Dynamic-Programming Pattern Clinics

The core chapter covers linear, grid, knapsack, sequence, edit, and stock state. Word break makes prefix feasibility concrete, while matrix-chain multiplication turns the interval-DP boundary into a complete derivation.

Clinic 1: word break as prefix feasibility

Define:

```text
reachable[end] = text[0..end) can be segmented into dictionary words
```

Base case `reachable[0] = true` represents the empty prefix. For every reachable start, try word endings up to the maximum dictionary word length. A matching word makes that end reachable.

This is not plain greedy. Choosing the longest or shortest matching word can block a valid later segmentation. DP retains all reachable prefix boundaries without materializing every complete sentence.

If n is the text length and L is the maximum dictionary word length, the state/transition structure is O(nL), excluding the cost of substring creation and hashing. Java substring creates a new string in current mainstream JDKs, so a production implementation may use a trie, region comparison, or indexes to control allocation.

Clinic 2: matrix-chain interval DP

Matrices are described by dimensions `d[0..n]`; matrix i has shape `d[i] x d[i+1]`. Multiplication order changes scalar work but not the final matrix.

State:

```text
cost[left][right] = minimum scalar multiplications for matrices left through right
```

Base: one matrix needs zero multiplications. For every split between `left` and `right`:

```text
cost[left][split]
+ cost[split + 1][right]
+ d[left] * d[split + 1] * d[right + 1]
```

Evaluate shorter intervals before longer intervals. There are O(n squared) states and O(n) split choices per state, so time is O(n cubed) and table space is O(n squared).

For dimensions `[40, 20, 30, 10, 30]`, the minimum cost is 26,000. Greedily multiplying the currently cheapest adjacent pair is not generally safe because each merge changes future dimensions.

Runnable Java 21 clinic

JAVA (continued 1/3)

```java
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public final class DynamicProgrammingCoverageClinic {
    private DynamicProgrammingCoverageClinic() {
    }

    public static boolean canSegment(String text, Set<String> dictionary) {
        Objects.requireNonNull(text, "text");
        Objects.requireNonNull(dictionary, "dictionary");
        Set<String> words = new HashSet<>();
        int maximumLength = 0;
        for (String word : dictionary) {
            if (word == null || word.isEmpty()) {
                throw new IllegalArgumentException("dictionary words must be nonempty");
            }
            words.add(word);
            maximumLength = Math.max(maximumLength, word.length());
        }

        boolean[] reachable = new boolean[text.length() + 1];
        reachable[0] = true;
        for (int start = 0; start < text.length(); start++) {
            if (!reachable[start]) {
                continue;
```

JAVA (continued 2/3)

```
            }
            int lastEnd = Math.min(text.length(), start + maximumLength);
            for (int end = start + 1; end <= lastEnd; end++) {
                if (words.contains(text.substring(start, end))) {
                    reachable[end] = true;
                }
            }
        }
        return reachable[text.length()];
    }

    public static long minimumMatrixChainCost(int[] dimensions) {
        Objects.requireNonNull(dimensions, "dimensions");
        if (dimensions.length < 2) {
            throw new IllegalArgumentException("at least one matrix is required");
        }
        for (int dimension : dimensions) {
            if (dimension <= 0) {
                throw new IllegalArgumentException("dimensions must be positive");
            }
        }

        int matrices = dimensions.length - 1;
        long[][] cost = new long[matrices][matrices];
        for (int length = 2; length <= matrices; length++) {
```

JAVA (continued 3/3)

```
            for (int left = 0; left + length <= matrices; left++) {
                int right = left + length - 1;
                cost[left][right] = Long.MAX_VALUE;
                for (int split = left; split < right; split++) {
                    long multiply = Math.multiplyExact(
                            Math.multiplyExact((long) dimensions[left],
                                    dimensions[split + 1]),
                            dimensions[right + 1]);
                    long candidate = Math.addExact(
                            Math.addExact(cost[left][split], cost[split + 1][right]),
                            multiply);
                    cost[left][right] = Math.min(cost[left][right], candidate);
                }
            }
        }
        return cost[0][matrices - 1];
    }

    public static void main(String[] args) {
        assert canSegment("leetcode", Set.of("leet", "code"));
        assert !canSegment("catsandog", Set.of("cats", "dog", "sand", "and", "cat"));
        assert minimumMatrixChainCost(new int[] {40, 20, 30, 10, 30}) == 26_000;
        System.out.println("PASS essential dynamic-programming clinics");
    }
}
```

Expected output with assertions enabled:

TEXT

```
PASS essential dynamic-programming clinics
```

Interviewer follow-up chain with model answers

Interviewer: How would you return one word-break segmentation?

Candidate: Store the predecessor start whenever an end first becomes reachable. If the final index is reachable, walk predecessors backward and reverse the words. If a specific optimal segmentation is required, the state must encode that objective.

Interviewer: Why is matrix-chain DP evaluated by increasing interval length?

Candidate: Every transition reads two strict subintervals. Increasing length is a topological order of that dependency graph.

Interviewer: Can matrix-chain storage be compressed to one dimension?

Candidate: Not by the simple rolling technique used for linear DP. Many nonadjacent subinterval states remain live for larger intervals. If only the cost is needed, specialized optimizations require additional mathematical structure; they are not a generic compression rule.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Dynamic Programming Interview Rounds

Round 1: minimum coins for an amount

Prompt

Given positive coin denominations with unlimited reuse, return the fewest coins needed to form `amount`, or -1.

Candidate derivation

State: `dp[a]` is the minimum coins required to form exact amount `a`; unreachable states use an infinity sentinel. Base `dp[0] = 0`. For each amount, try every coin that can be the final coin.

JAVA

```java
static int minimumCoins(int[] coins, int amount) {
    if (amount < 0) throw new IllegalArgumentException("negative amount");
    int unreachable = amount + 1;
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, unreachable);
    dp[0] = 0;
    for (int current = 1; current <= amount; current++) {
        for (int coin : coins) {
            if (coin <= 0) throw new IllegalArgumentException("positive coins required");
            if (coin <= current && dp[current - coin] != unreachable) {
                dp[current] = Math.min(dp[current], dp[current - coin] + 1);
            }
        }
    }
    return dp[amount] == unreachable ? -1 : dp[amount];
}
```

Follow-up answers

Why `amount + 1` as infinity? With positive integer coins, any feasible solution uses at most `amount` coins when denomination 1 exists, and the sentinel avoids overflow from `Integer.MAX_VALUE + 1`. Without coin 1, it remains safely above every possible coin count.

Need the actual coins? Store the last chosen coin for each improved amount and walk backward. Define tie-breaking among equal-length solutions.

Complexity? O(amount * numberOfCoins) time and O(amount) space; pseudo-polynomial in numeric amount.

Round 2: longest common subsequence

Prompt

Return the length of the longest sequence appearing in order, not necessarily contiguously, in both strings.

State and transition

TEXT

```
dp[i][j] = LCS length of first i chars of first and first j chars of second
```

If final characters match, extend `dp[i-1][j-1]`. Otherwise discard one final character and keep the better of `dp[i-1][j]` and `dp[i][j-1]`.

JAVA

```java
static int lcsLength(String first, String second) {
    int[][] dp = new int[first.length() + 1][second.length() + 1];
    for (int i = 1; i <= first.length(); i++) {
        for (int j = 1; j <= second.length(); j++) {
            if (first.charAt(i - 1) == second.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[first.length()][second.length()];
}
```

Follow-up answers

Substring instead? Contiguous matching resets on mismatch; the state and answer aggregation differ.

Space compression? Length needs only previous and current rows: O(min(m,n)) space after choosing the shorter string as columns. Reconstruction generally needs the full table or a divide-and-conquer method.

Unicode? `charAt` compares UTF-16 code units. If the contract is code-point sequences, convert accordingly; user-perceived grapheme sequences need a stronger text boundary.

Round 3: equal-subset partition

Prompt

Return whether positive integers can be partitioned into two groups with equal sum.

Candidate derivation

The total must be even. The problem becomes 0/1 subset sum for target `total / 2`.

JAVA

```java
static boolean canPartition(int[] values) {
    long total = 0L;
    for (int value : values) {
        if (value < 0) throw new IllegalArgumentException("nonnegative values required");
        total += value;
    }
    if ((total & 1L) != 0L || total / 2L > Integer.MAX_VALUE) return false;
    int target = (int) (total / 2L);
    boolean[] possible = new boolean[target + 1];
    possible[0] = true;
    for (int value : values) {
        for (int sum = target; sum >= value; sum--) {
            possible[sum] |= possible[sum - value];
        }
    }
    return possible[target];
}
```

Follow-up answers

Why downward? Each value may be used once. Upward iteration could read a state written earlier in the same item iteration and reuse that item.

Large values? O(n * target) may be infeasible even for moderate n. A meet-in-the-middle approach can suit small n with huge values; a bitset can improve constants for bounded sums; approximation depends on the business contract.

Closing answer pattern

Say the state sentence, choices, recurrence, bases, evaluation order, answer location, state-count complexity, numeric sentinel policy, reconstruction plan, and whether compression preserves required information.

SDE-2 CORE CHAPTER

Chapter 5 - State, Transitions, and Reconstruction

Dynamic programming is organized reuse of overlapping subproblems. The table is not the starting point. First write what one state means, what earlier states it depends on, and why evaluation order makes those dependencies available. Then choose memoization, tabulation, or space optimization.

The complete Java 21 implementations are in `DynamicProgrammingInterviewChecks.java`.

The five-line DP design

For every problem, write:

1. **State:** what does `dp[...]` mean?
2. **Transition:** which smaller states produce it?
3. **Base:** which states are known directly?
4. **Order:** when is every dependency ready?
5. **Answer:** which state or aggregate is returned?

Complexity is number of reachable states times work per state, plus reconstruction/output.

Memoization versus tabulation

Minimum coins state:

```text
dp[a] = minimum coins needed to make amount a, or unreachable
dp[0] = 0
dp[a] = 1 + min(dp[a-coin]) over legal reachable predecessors
```

Top-down memoization follows only requested states and resembles the recurrence. Bottom-up tabulation avoids recursion depth and has predictable iteration. The companion implements both and compares them on random inputs.

Memo needs three states in its cache: unknown, unreachable, and a nonnegative answer. Conflating unknown with unreachable can either recompute endlessly or skip valid work.

The teaching memoized coin method limits amount because a valid recurrence can still create unsafe Java call depth. Tabulation is the practical choice for larger linear amount domains.

Space optimization comes after dependency analysis

Fibonacci depends on only two previous values, so a full array is unnecessary:

```text
previous = F(i-2)
current  = F(i-1)
next     = previous + current
```

The companion caps `n` at 92 because `F(93)` exceeds signed `long`. It does not compute one unnecessary next value after obtaining `F(92)`.

For a 2D DP, one row may be enough only if current-row updates do not overwrite a value still needed. Loop direction is part of the state transition.

0/1 knapsack and reconstruction

State:

```text
best[i][c] = maximum value using first i items with capacity c
```

Transition excludes item `i-1`, or includes it once from the previous row:

```text
best[i][c] = best[i-1][c]
if weight <= c:
    best[i][c] = max(best[i][c], best[i-1][c-weight] + value)
```

Using the previous row is what enforces 0/1 usage. A one-row optimization must iterate capacity downward; upward iteration would allow the same item repeatedly and solve unbounded knapsack instead.

To reconstruct, walk backward. If `best[i][c] != best[i-1][c]`, item `i-1` was chosen under the companion's tie policy; record it and subtract its weight. Ties are deliberately resolved by exclusion, so multiple optimal sets can exist while output remains deterministic.

JAVA

```java
import java.util.*;

record Item(String id, int weight, int value) { }

record Packing(int value, List<Item> chosen) { }

static Packing knapsack(List<Item> items, int capacity) {
    int n = items.size();
    int[][] best = new int[n + 1][capacity + 1];

    for (int i = 1; i <= n; i++) {
        Item item = items.get(i - 1);
        for (int c = 0; c <= capacity; c++) {
            best[i][c] = best[i - 1][c];                    // exclude
            if (item.weight() <= c) {
                int include = best[i - 1][c - item.weight()] + item.value();
                if (include > best[i][c]) {                 // strict: ties exclude
                    best[i][c] = include;
                }
            }
        }
    }

    List<Item> chosen = new ArrayList<>();
    int c = capacity;
    for (int i = n; i > 0; i--) {
        if (best[i][c] != best[i - 1][c]) {                 // this row improved on the last
            Item item = items.get(i - 1);
            chosen.add(item);
            c -= item.weight();
        }
    }
    Collections.reverse(chosen);                            // back to input order
    return new Packing(best[n][capacity], List.copyOf(chosen));
}
```

The `>` in `include > best[i][c]` is a **tie policy, not a correctness requirement**, and it is worth being precise about which. Switching it to `>=` does not break reconstruction: on a tie the stored value is unchanged, so `best[i][c] != best[i-1][c]` is still false, the walk-back still follows the exclude path, and that path is equally optimal. Tested both ways over 4,000 random instances seeded with duplicate values - zero inconsistencies either way.

What the strict comparison buys is **determinism**: it fixes *which* optimal set you report when several exist. That matters for tests, for diffable output, and for a reviewer asking why the answer changed. Say "ties resolve by exclusion" rather than implying the program would otherwise be wrong.

The genuine correctness constraint is elsewhere: the transition must read `best[i - 1][...]`, the **previous** row. Reading the current row would let one item be taken repeatedly and quietly solve unbounded knapsack instead.

Reconstruction needs the **full table**, which is why the one-row space optimisation and the ability to report the chosen set are mutually exclusive. Say that trade-off out loud before optimising: `O(n x capacity)` memory buys you an answer you can explain, and interviewers usually want the set.

Checked against exhaustive subset enumeration over 600 random instances, asserting three things rather than one: the value matches the brute-force optimum, the reconstructed set fits within capacity, and its values sum to exactly the reported optimum. Zero failures.

Edit distance: define the prefix state

```text
dp[i][j] = edits to convert first i chars of first string
           into first j chars of second string
```

Base row/column are insert/delete counts. Equal final code units take diagonal unchanged. Otherwise add one to the minimum of:

- diagonal: replace;
- above: delete from first; and
- left: insert into first.

The companion keeps two rows and makes the shorter string the column dimension, using `O(min(m,n))` space. It operates on UTF-16 `char` units. A code-point or grapheme-aware product needs a different tokenization contract.

Grid path: identity and unreachable sentinels

For movement right/down, each cell depends on above and left. A one-row array holds the best path to the previous row before update and the current row to the left after update.

Initialize all entries unreachable except a synthetic zero before the start. Add with `Math.addExact` so numeric overflow does not silently become a better path. Reject jagged/empty grids under the rectangular contract.

Obstacles require an explicit unreachable state; do not use zero if zero is a valid path cost.

Stock state machine

At most two transactions can be expressed as four best states after each price:

```text
buyFirst   = best balance after first buy
sellFirst  = best balance after first sell
buySecond  = best balance after second buy
sellSecond = best balance after second sell
```

Updating in transaction order is safe when buying and selling on the same day is allowed and adds zero profit. Each state summarizes all histories ending in that phase; no day-by-day transaction list is stored.

This is DP even though the table is four variables. State compression does not turn it into greedy.

Validate with independent oracles

The companion compares:

- memoized and tabulated coin change across deterministic random inputs; and
- knapsack DP with exhaustive subset enumeration on small random instances.

An oracle should be simpler and structurally different. Comparing two copies of the same recurrence can preserve the same bug.

Edge-case matrix

Case	Correct handling	Common failure
amount zero	zero coins, even with empty coin set	requiring one coin
nonpositive coin	reject	self/cyclic dependency
unreachable amount	distinct sentinel/result	adding one to infinity
memo unknown versus impossible	separate markers	repeated work/wrong pruning
large recursion depth	tabulate or cap	stack overflow
Fibonacci 92/93	92 fits, 93 rejected for `long`	compute overflowed extra term
0/1 one-row knapsack	capacity decreases	accidental item reuse
multiple optimal item sets	state tie policy	claiming unique reconstruction
empty string	base row/column	reading character `-1`
Unicode text	state code-unit contract	claiming user-perceived edits
jagged/empty grid	reject or define	invalid left/above access
negative stock price	reject domain input	nonsensical profit

Six live interview Q&A chains

1. State definition

Interviewer: Why is "dp of amount" too vague?

Candidate: It must say minimum coins for exactly that amount and how impossible states are represented. Without exact meaning, transition and answer cannot be proved.

2. Memo marker

Interviewer: Can `-1` mean both unknown and unreachable?

Candidate: Not safely. An impossible state would look uncomputed and be explored repeatedly. I use a separate unknown marker, then cache `-1` as the computed impossible result.

3. Knapsack loop direction

Interviewer: Why iterate capacity downward in one-row 0/1 knapsack?

Candidate: So `dp[c-weight]` still belongs to the previous item set. Upward order may read a value written by the current item and choose that item multiple times.

4. Reconstruction

Interviewer: Does a maximum value prove which items were selected?

Candidate: No. I retain enough table state to walk backward and apply a deterministic tie rule. A value-only compressed DP cannot always reconstruct without additional decisions or recomputation.

5. Space optimization

Interviewer: Why not always compress to one row?

Candidate: It can destroy information needed for reconstruction and is unsafe if current updates overwrite future dependencies. I first prove dependency direction, then decide whether memory or explainability matters more.

6. DP versus greedy

Interviewer: Coin change feels greedy. Why use DP?

Candidate: Arbitrary coin systems lack a safe largest-coin exchange proof; `[1,3,4]` for amount 6 is a counterexample. DP keeps the amount as state and evaluates every last-coin choice, guaranteeing the optimum under the stated bounds.

Run the companion

BASH

```
javac --release 21 -Xlint:all -Werror DynamicProgrammingInterviewChecks.java
java DynamicProgrammingInterviewChecks
```

Expected final line: `PASS 20 dynamic-programming checks`.

SDE-2 CORE CHAPTER

Chapter 6 - Subsequence and Palindrome DP

Why this chapter exists

Subsequence problems are the densest cluster in the dynamic-programming interview space. Longest increasing subsequence, longest common subsequence, longest palindromic subsequence, palindromic substring counting, and their many disguises account for a large share of DP prompts, and they share a small number of state shapes.

They also contain the two most common derivation errors. The first is conflating **subsequence** with **substring**: one allows gaps, the other does not, and the state definitions are not interchangeable. The second is reaching for the O(n log n) longest-increasing-subsequence algorithm without being able to say what its auxiliary array actually holds - which collapses the moment an interviewer asks for the subsequence itself rather than its length.

This chapter derives each from the six-question protocol in chapter 1, and pays particular attention to reconstruction, because "return the answer, not just its size" is the standard follow-up.

Subsequence versus substring

State the distinction before writing any state, because every later decision depends on it.

```text
"interview"
  substring    (contiguous)       "terv"   yes
                                  "itv"    no
  subsequence  (order preserved)  "itv"    yes
                                  "vti"    no
```

The consequence for DP is structural:

- **Substring** state is usually an interval `[i, j]` or a run ending at `i`. Extending means adding one adjacent character, so transitions look at `i-1` or `j+1`.
- **Subsequence** state is usually a pair of prefix lengths, or a position plus a comparison predicate. Extending means *choosing whether to include* the next character, so transitions branch on take or skip.

If a candidate writes `dp[i]` = "longest palindromic substring ending at i", they have already lost, because palindromes are not built by extension at one end. The correct substring state is the interval.

Longest common subsequence, and what it generalizes

LCS is the archetype. Both inputs are consumed from the front, and each step either matches both characters or discards one.

The six questions:

1. *State:* `dp[i][j]` = length of the LCS of `a[0,i)` and `b[0,j)`. Prefix lengths, not indices - this is what makes the base case free.
2. *Transition:* if `a[i-1] == b[j-1]` then `dp[i][j] = 1 + dp[i-1][j-1]`, else `max(dp[i-1][j], dp[i][j-1])`.
3. *Base:* `dp[0][j] = dp[i][0] = 0`. An empty prefix shares nothing.
4. *Order:* increasing `i`, then increasing `j`. Each cell reads only smaller indices.
5. *Answer:* `dp[n][m]`.
6. *Complexity:* O(n*m) time, O(*n*m) space, reducible to O(min(n, m)) if reconstruction is not required.

The half-open prefix convention matters. With `dp[i][j]` meaning "first `i` and first `j` characters", index `0` is the empty prefix and no special-casing is needed. Defining state as "ending at index i" forces an awkward base row and is the usual source of off-by-one bugs.

Reconstruction walks backward from `(n, m)`:

TEXT

```
while i > 0 and j > 0:
    if a[i-1] == b[j-1]:  emit a[i-1]; i--; j--
    elif dp[i-1][j] >= dp[i][j-1]:  i--
    else:  j--
reverse the emitted characters
```

The tie-break on the `elif` decides *which* LCS you return when several are optimal. Interviewers sometimes ask for a specific one - lexicographically smallest, say - and the honest answer is that the tie-break policy, not the recurrence, controls it.

What LCS generalizes to

Problem	Reduction
Longest palindromic subsequence	LCS of `s` and `reverse(s)`
Shortest common supersequence	`n + m - LCS(a, b)`
Minimum deletions to make equal	`(n - LCS) + (m - LCS)`
Edit distance (no substitution)	Same as minimum deletions
Diff / patch output	LCS reconstruction with the non-matching parts emitted

The palindromic-subsequence reduction is worth internalizing because it converts an unfamiliar problem into one you have already derived. It is correct because a subsequence of `s` that is also a subsequence of `reverse(s)` reads the same in both directions.

Longest palindromic subsequence, derived directly

The reduction above works, but interviewers often want the interval formulation, and it is the one that extends to counting.

1. *State:* `dp[i][j]` = length of the longest palindromic subsequence within `s[i..j]` inclusive.
2. *Transition:* if `s[i] == s[j]` then `dp[i][j] = 2 + dp[i+1][j-1]`, else `max(dp[i+1][j], dp[i][j-1])`.
3. *Base:* `dp[i][i] = 1`; every single character is a palindrome of length one.
4. *Order:* **by increasing interval length**, not by increasing `i`. `dp[i][j]` reads `dp[i+1][j-1]`, which is a shorter interval.
5. *Answer:* `dp[0][n-1]`.
6. *Complexity:* O(n^2) time and space.

Step 4 is the one candidates get wrong. A naive nested loop over `i` ascending and `j` ascending reads cells that have not been computed. Two correct orders exist: iterate over interval length, or iterate `i` **descending** and `j` ascending. Both guarantee that `i+1` and `j-1` are already final.

TEXT

```
length = 1:  [0,0] [1,1] [2,2] [3,3]      all base
length = 2:  [0,1] [1,2] [2,3]            reads length-0
length = 3:  [0,2] [1,3]                  reads length-1
length = 4:  [0,3]                        reads length-2
```

Palindromic substrings: expand versus DP

Counting palindromic *substrings* has two standard solutions, and knowing why the simpler one is usually better is the interview signal.

Interval DP. `isPal[i][j]` is true when `s[i] == s[j]` and (`j - i < 2` or `isPal[i+1][j-1]`). O(n^2) time and O(n^2) space.

Expand around centres. Every palindrome has a centre - a character for odd lengths, a gap for even. There are `2n - 1` centres; expand outward from each while the characters match. O(n^2) time and **O(1) space**.

JAVA

```java
int countPalindromicSubstrings(String s) {
    int total = 0;
    for (int centre = 0; centre < 2 * s.length() - 1; centre++) {
        int left = centre / 2;
        int right = left + centre % 2;          // odd centre: right == left
        while (left >= 0 && right < s.length()
                && s.charAt(left) == s.charAt(right)) {
            total++;
            left--;
            right++;
        }
    }
    return total;
}
```

Same asymptotic time, quadratically less space, and considerably less code. Reach for the DP table only when you need the `isPal` relation itself for a *later* DP - palindrome partitioning being the standard case, where `isPal` is precomputed and then a second DP minimizes cuts.

> **When O(n) is required:** Manacher's algorithm finds the longest palindromic substring in linear time. It is rarely expected at SDE-2 and is a poor use of interview minutes unless asked for explicitly. Knowing it exists, and that it works by reusing mirror information around a rightmost boundary, is usually sufficient.

Longest increasing subsequence: both algorithms, honestly

The O(n^2) formulation

1. *State:* `dp[i]` = length of the longest increasing subsequence **ending at index i**.
2. *Transition:* `dp[i] = 1 + max(dp[j])` over all `j < i` with `a[j] < a[i]`; `1` if none.
3. *Base:* implicit - every element alone is a subsequence of length one.
4. *Order:* increasing `i`.
5. *Answer:* `max(dp)` - note **not** `dp[n-1]`, because the LIS need not end at the last element.
6. *Complexity:* O(n^2) time, O(n) space.

Reconstruction stores a predecessor index whenever `dp[i]` improves, then follows predecessors back from the argmax. This version reconstructs easily, which is why it deserves to be derived first.

The O(n log n) formulation, and what the array actually holds

This is where candidates lose points by reciting an algorithm they cannot explain.

Maintain an array `tails`, where `tails[k]` **is the smallest possible tail value of an increasing subsequence of length** `k+1` seen so far. It is not the subsequence. It is not sorted input. It is a set of best-case endings, and it is provably increasing.

JAVA

```java
int lengthOfLIS(int[] values) {
    int[] tails = new int[values.length];
    int size = 0;
    for (int value : values) {
        int position = Arrays.binarySearch(tails, 0, size, value);
        if (position < 0) {
            position = -(position + 1);         // insertion point
        }
        tails[position] = value;                // extend, or improve a tail
        if (position == size) {
            size++;
        }
    }
    return size;
}
```

Two facts make this correct:

- `tails` is strictly increasing, so binary search is valid. A shorter subsequence can always end no higher than a longer one.
- Overwriting `tails[position]` never shortens the answer. It records a subsequence of the same length ending at a smaller value, which can only make future extensions easier.

The critical honesty: `tails` is not itself a valid subsequence. Printing it as the answer is wrong, and interviewers ask exactly this. To reconstruct, record for each input element the index it was placed at, plus the predecessor - the element then at `tails[position - 1]` - and walk back from the last element that reached the maximum length.

Strict versus non-decreasing is controlled by the search, not the recurrence. `binarySearch` for the leftmost position gives strictly increasing; searching for the rightmost insertion point (upper bound) allows equal values. Getting this backwards silently changes the answer, and it is a common follow-up.

TRACE IT | Worked example: longest palindromic subsequence with reconstruction

JAVA (continued 1/3)

```java
import java.util.Arrays;

public final class PalindromicSubsequence {

    /** Length of the longest palindromic subsequence of s. */
    static int length(String s) {
        int n = s.length();
        if (n == 0) {
            return 0;
        }
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }
        // Interval length ascending: dp[i][j] reads the shorter dp[i+1][j-1].
        for (int span = 2; span <= n; span++) {
            for (int i = 0; i + span - 1 < n; i++) {
                int j = i + span - 1;
                dp[i][j] = s.charAt(i) == s.charAt(j)
                        ? 2 + (span == 2 ? 0 : dp[i + 1][j - 1])
                        : Math.max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
        return dp[0][n - 1];
    }

    /** One longest palindromic subsequence, rebuilt from the same table. */
    static String reconstruct(String s) {
```

JAVA (continued 2/3)

```
        int n = s.length();
        if (n == 0) {
            return "";
        }
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }
        for (int span = 2; span <= n; span++) {
            for (int i = 0; i + span - 1 < n; i++) {
                int j = i + span - 1;
                dp[i][j] = s.charAt(i) == s.charAt(j)
                        ? 2 + (span == 2 ? 0 : dp[i + 1][j - 1])
                        : Math.max(dp[i + 1][j], dp[i][j - 1]);
            }
        }

        StringBuilder head = new StringBuilder();
        int i = 0;
        int j = n - 1;
        String middle = "";
        while (i <= j) {
            if (i == j) {
                middle = String.valueOf(s.charAt(i));
                break;
            }
            if (s.charAt(i) == s.charAt(j)) {
                head.append(s.charAt(i));
```

JAVA (continued 3/3)

```
                i++;
                j--;
            } else if (dp[i + 1][j] >= dp[i][j - 1]) {
                i++;                                // tie-break: prefer moving i
            } else {
                j--;
            }
        }
        // Built in three explicit steps. Writing this as
        // `head + middle + head.reverse().toString()` also works, but only
        // because Java evaluates operands left to right and stringifies the
        // first `head` before `reverse()` mutates it. Correctness that depends
        // on evaluation order next to in-place mutation is not worth the line
        // it saves.
        String firstHalf = head.toString();
        String secondHalf = new StringBuilder(firstHalf).reverse().toString();
        return firstHalf + middle + secondHalf;
    }

    public static void main(String[] args) {
        String s = "bbbab";
        System.out.println(length(s));        // 4
        System.out.println(reconstruct(s));   // bbbb
        System.out.println(length("cbbd"));   // 2
        System.out.println(reconstruct("cbbd")); // bb
        System.out.println(length(""));       // 0
    }
}
```

Note the `span == 2` guard. Without it, `dp[i+1][j-1]` on a two-character interval indexes `dp[i+1][i]`, which is below the diagonal and holds a meaningless zero. It happens to be zero here so the code would still work, but relying on that is exactly the kind of accident that breaks when the base case changes.

The reconstruction returns the first half, the optional middle character, then the reversed first half - which is why `head` is reversed in place at the end.

WATCH OUT | Edge cases and common mistakes

- Defining a substring state as "ending at i" for palindromes, which does not extend.
- Iterating `i` ascending for interval DP, reading cells not yet computed. Use interval length, or `i` descending.
- Returning `dp[n-1]` for LIS instead of `max(dp)`.
- Presenting the `tails` array as the longest increasing subsequence. It is not one.
- Confusing strict and non-decreasing LIS by using the wrong binary-search boundary.
- Reaching for O(n log n) LIS when the follow-up needs reconstruction, then being unable to produce it.
- Building an O(n^2) `isPal` table when expand-around-centre solves it in O(1) space.
- Forgetting that the palindromic-subsequence-to-LCS reduction needs `reverse(s)`, not a sort.
- Off-by-one from mixing inclusive `[i..j]` interval state with half-open `[0,i)` prefix state in the same solution. Pick one convention per problem and say which.
- Returning any optimal answer when the prompt asked for a specific tie-break, without noting that the policy is a choice.

TRY IT | Interview questions and model answers

What is the difference between a subsequence and a substring, and why does it change the DP?

A substring is contiguous; a subsequence preserves order but allows gaps. Substring state is typically an interval or a run ending at an index, because extension adds an adjacent character. Subsequence state is typically a pair of prefix lengths, because each step chooses to include or skip. The state shapes are not interchangeable, and most derivation errors start by picking the wrong one.

Derive the longest palindromic subsequence.

`dp[i][j]` over the inclusive interval `s[i..j]`. Matching ends contribute two plus the inner interval; otherwise take the better of dropping either end. Base is one for every single character. Evaluate by increasing interval length, because the transition reads a shorter interval - iterating `i` ascending reads uncomputed cells. Alternatively, it is exactly `LCS(s, reverse(s))`.

What does the `tails` array in O(n log n) LIS actually contain?

`tails[k]` is the smallest tail value among all increasing subsequences of length `k+1` found so far. It is strictly increasing, which is what makes binary search valid, but it is not itself a subsequence of the input.

Reconstructing the actual subsequence requires recording each element's placement index and predecessor separately.

Count palindromic substrings. Which approach and why?

Expand around each of the `2n-1` centres. Same O(n^2) time as the interval DP but O(1) space and far less code. I would only build the `isPal` table if a later DP consumes it - palindrome partitioning, for instance, where you precompute `isPal` then minimize cuts.

How do you return the subsequence rather than its length?

For O(n^2) LIS, store a predecessor index whenever `dp[i]` improves, then follow predecessors from the argmax. For LCS, walk backward from `dp[n][m]` taking a diagonal on a match and otherwise moving toward the larger neighbour. In both cases the tie-break policy determines which optimal answer you return, and that is a decision to state rather than leave implicit.

Your LIS returns the wrong answer for equal adjacent values. What happened?

The binary-search boundary. Searching for the leftmost insertion point enforces strictly increasing; searching for the upper bound permits equal values and yields the non-decreasing variant. The recurrence is unchanged - only the search boundary moves - which is why the bug is easy to introduce and hard to spot.

TRY IT | Exercises

1. **Foundation:** For `"character"`, list three subsequences that are not substrings and one substring that is not a palindrome.
2. **Foundation:** Write the six-question derivation for longest common subsequence before writing any code.
3. **Interview Core:** Implement LCS with reconstruction. Then change the tie-break and show a pair of inputs where the returned subsequence differs while the length does not.
4. **Interview Core:** Implement longest palindromic subsequence twice - once as an interval DP, once as `LCS(s, reverse(s))` - and verify both agree over a few hundred random strings.
5. **Interview Core:** Implement the interval DP with `i` ascending and demonstrate the wrong answer it produces. Explain exactly which cell was read before it was written.
6. **Interview Core:** Count palindromic substrings by expanding around centres, then by interval DP, and compare peak memory on a 10,000-character input.
7. **Interview Core:** Implement O(n^2) LIS with reconstruction, then O(n log n) LIS with reconstruction. State what extra bookkeeping the second needs.
8. **SDE-2 Follow-up:** Convert your LIS to the non-decreasing variant by changing only the search boundary. Write the test that distinguishes them.
9. **SDE-2 Follow-up:** Solve palindrome partitioning with minimum cuts using a precomputed `isPal` table, and justify why the table earns its space here when it did not for counting.
10. **Challenge:** Given the LCS table for two strings, count how many distinct longest common subsequences exist. Explain why this is not simply the number of backward paths.

RECAP | Chapter summary

Subsequence problems share a small set of state shapes, and picking the wrong one is the first and most expensive error - substring state extends at an end, subsequence state chooses take-or-skip over prefixes. LCS with half-open prefix state is the archetype, and it generalizes to longest palindromic subsequence, shortest common supersequence, and minimum deletions by reduction rather than by new derivation. Interval DP for palindromes must be evaluated by increasing interval length, because the transition reads a shorter interval; iterating `i` ascending reads uncomputed cells and is the standard bug. For counting palindromic substrings, expanding around the `2n-1` centres matches the DP's time in constant space, so the table only earns its place when a later DP consumes it. Longest increasing subsequence deserves both derivations: the O(n^2) version reconstructs naturally, while the O(n log n) version depends on knowing that `tails[k]` holds the smallest tail of a length-`k+1` subsequence rather than a subsequence itself - and on knowing that the strict versus non-decreasing distinction lives entirely in the binary-search boundary.

TRY IT | Revision checklist

- [ ] I can state the substring/subsequence distinction and its effect on state shape.
- [ ] I can derive LCS with half-open prefix state and reconstruct from the table.
- [ ] I know the four problems LCS reduces to.
- [ ] I can derive longest palindromic subsequence as interval DP and as an LCS reduction.
- [ ] I know why interval DP must iterate by span, and which cell breaks if it does not.
- [ ] I can count palindromic substrings in O(1) space and say when the table is worth it.
- [ ] I can implement O(n^2) LIS with reconstruction from predecessors.
- [ ] I can explain precisely what `tails[k]` holds and why binary search is valid.
- [ ] I can reconstruct the actual subsequence from the O(n log n) algorithm.
- [ ] I can switch between strict and non-decreasing LIS and test the difference.

SDE-2 CORE CHAPTER

Chapter 7 - Bitmask, Digit, and Bounded-State DP

Why this chapter exists

The DP problems covered so far all had a state that was obviously an index or a pair of indices. This chapter covers the cases where the state is a **set**, a **position within a numeral**, or a **budget** - shapes that are less obvious but appear regularly once an interviewer starts pushing past the standard catalogue.

They share one property worth naming immediately: each is a technique for making an exponential search space finite and enumerable. Bitmask DP does not make an NP-hard problem polynomial. It makes `n!` into `2^n * n`, which is a genuine and useful difference at `n = 20` and no help at all at `n = 60`. Being able to say that precisely is the interview signal.

Bitmask DP: when the state is a subset

Recognizing it

Reach for a bitmask when **all** of the following hold:

- The state must record *which* elements have been used, not merely how many.
- Order of selection does not matter for the state, only the set does.
- `n` is small - roughly 20 or fewer, occasionally 24 with tight memory.

That last condition is a hard gate. `2^20` is about a million states, which is comfortable; `2^25` is 33 million, which is usually not. If the prompt says `n <= 20`, the constraint is telling you the intended solution.

The representation

An `int` holds the subset. Bit `k` set means element `k` is used.

JAVA

```java
int full = (1 << n) - 1;          // all n elements used
boolean used = (mask & (1 << k)) != 0;
int with = mask | (1 << k);       // add element k
int without = mask & ~(1 << k);   // remove element k
int count = Integer.bitCount(mask);
```

`Integer.bitCount` is worth knowing by name: it compiles to a single `POPCNT` instruction on modern hardware, so using it as a loop counter is free.

Worked derivation: minimum-cost assignment

n tasks, n workers, `cost[worker][task]`*. Assign each worker exactly one task, minimizing total cost.*

The naive search tries all `n!` assignments. The insight is that once you have assigned the first `k` workers, **which** tasks remain is all that matters - not the order they were assigned in. That collapses `n!` orderings into `2^n` subsets.

1. *State:* `dp[mask]` = minimum cost to assign tasks in `mask` to the first `bitCount(mask)` workers.
2. *Transition:* let `worker = bitCount(mask)`. For each unused task `t`, `dp[mask | (1<<t)] = min(that, dp[mask] + cost[worker][t])`.
3. *Base:* `dp[0] = 0`; no workers assigned, no cost.
4. *Order:* increasing `mask` as an integer. Adding a bit strictly increases the value, so every predecessor is already final.
5. *Answer:* `dp[full]`.
6. *Complexity:* O(2^n * n) time, O(2^n) space.

Step 4 is the elegant part and worth stating explicitly in an interview: iterating masks in increasing numeric order is a valid topological order, because `mask | (1<<t) > mask` whenever bit `t` was unset. No explicit ordering logic is needed.

JAVA

```java
static int minimumAssignmentCost(int[][] cost) {
    int n = cost.length;
    int full = (1 << n) - 1;
    int[] dp = new int[1 << n];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;

    for (int mask = 0; mask < full; mask++) {
        if (dp[mask] == Integer.MAX_VALUE) {
            continue;                    // unreachable, do not relax from it
        }
        int worker = Integer.bitCount(mask);
        for (int task = 0; task < n; task++) {
            if ((mask & (1 << task)) != 0) {
                continue;                // already assigned
            }
            int next = mask | (1 << task);
            int candidate = dp[mask] + cost[worker][task];
            if (candidate < dp[next]) {
                dp[next] = candidate;
            }
        }
    }
    return dp[full];
}
```

The `dp[mask] == MAX_VALUE` guard is not decoration. Without it, `dp[mask] + cost` overflows to a negative number and silently poisons the table - the classic sentinel-arithmetic bug from chapter 3, appearing here in a new costume.

Travelling salesman, and the honest complexity statement

The Held-Karp formulation adds "where am I now" to the state:

- *State:* `dp[mask][last]` = cheapest path visiting exactly the cities in `mask`, ending at `last`.
- *Transition:* `dp[mask | 1<<next][next] = min(that, dp[mask][last] + dist[last][next])`.
- *Complexity:* O(2^n * n^2) time, O(2^n * n) space.

At `n = 20` that is roughly 400 million operations and 20 million table entries - feasible but not comfortable. At `n = 25` it is out of reach. **This is still exponential.** Held-Karp is a large constant-factor and asymptotic improvement over `O(n!)` brute force, not a polynomial algorithm, and TSP remains NP-hard. Candidates who present bitmask DP as "solving" TSP are making a claim an interviewer will test.

Subset-sum enumeration

Iterating every subset of a mask is a standard idiom worth recognizing:

JAVA

```java
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    int complement = mask ^ sub;
    // ... consider splitting mask into sub and complement
}
```

Summed over all masks this is O(3^n), not O(4^n), because each element is in `sub`, in `complement`, or outside `mask` - three choices per element. That counting argument is a common follow-up question.

Digit DP: when the state is a position in a numeral

Recognizing it

Digit DP answers "how many integers in `[low, high]` satisfy property P?" when the range is far too large to enumerate - typically up to 10^18. The state walks the decimal representation left to right.

The standard reduction is `count(high) - count(low - 1)`, so only a "count up to N" routine is needed.

The tight flag is the whole technique

Building a number digit by digit, at each position you may place any digit `0..9` - **unless** every previous digit exactly matched the corresponding digit of `N`, in which case you are bounded above by `N`'s digit at this position.

That single boolean is what makes the state finite:

- *State:* `(position, tight, ...problem-specific state)`.
- `position` - index into the digit string, 0 to 18.
- `tight` - whether the prefix so far equals `N`'s prefix.
- Extra state depends on the property: digit sum so far, last digit placed, count of a particular digit, remainder modulo k.

JAVA

```java
static long countUpTo(String digits, int position, boolean tight,
                      int state, Long[][][] memo) {
    if (position == digits.length()) {
        return isAccepting(state) ? 1 : 0;
    }
    if (!tight && memo[position][state][0] != null) {
        return memo[position][state][0];   // only cacheable when not tight
    }
    int limit = tight ? digits.charAt(position) - '0' : 9;
    long total = 0;
    for (int digit = 0; digit <= limit; digit++) {
        total += countUpTo(digits, position + 1,
                           tight && digit == limit,
                           nextState(state, digit), memo);
    }
    if (!tight) {
        memo[position][state][0] = total;
    }
    return total;
}
```

Memoize only when `tight` is false. This is the detail candidates miss. A tight state depends on the specific prefix of `N` and is visited at most once per position anyway, so caching it is both unnecessary and wrong if the cache is shared across different bounds.

A third flag, `started`, is often needed to handle leading zeros - "count numbers whose digits are strictly increasing" must not treat `007` as having leading digits that break the property.

Digit DP appears at SDE-2 occasionally rather than routinely. Recognizing the shape and naming the tight flag is usually enough; a full implementation under time pressure is a stretch goal.

Bounded-state DP: budgets, transactions, and cooldowns

The third shape adds a small bounded counter to an otherwise ordinary state. Stock problems are the canonical family, and they are worth working as a group because the progression shows how one extra dimension absorbs each new rule.

Variant	State	Size
Unlimited transactions	`dp[day][holding]`	2
At most one transaction	`dp[day][holding]` with no re-buy	2
At most k transactions	`dp[day][used][holding]`	2(k+1)
With cooldown	`dp[day][holding | cooling]`	3
With per-transaction fee	`dp[day][holding]`, fee on sell	2

The pattern to internalize: **a new rule usually becomes a new small dimension, not a new algorithm.** A candidate who has derived the two-state version can extend to k transactions in a sentence, and that fluency is what the interviewer is measuring.

JAVA

```java
static int maxProfitWithCooldown(int[] prices) {
    if (prices.length == 0) {
        return 0;
    }
    int holding = Integer.MIN_VALUE / 2;   // half-range: safe to add to
    int free = 0;                          // sold earlier, may buy
    int cooling = Integer.MIN_VALUE / 2;   // sold today, may not buy tomorrow

    for (int price : prices) {
        int previousHolding = holding;
        int previousFree = free;
        int previousCooling = cooling;

        holding = Math.max(previousHolding, previousFree - price);
        cooling = previousHolding + price;
        free = Math.max(previousFree, previousCooling);
    }
    return Math.max(free, cooling);
}
```

Two details carry the correctness. **Snapshot the previous values** before updating any of them - updating in place makes a later transition read this day's value instead of yesterday's, which silently permits an extra same-day trade. And `Integer.MIN_VALUE / 2` rather than `MIN_VALUE` gives an unreachable sentinel that can still be added to without overflowing, the same discipline as the bitmask guard above.

WATCH OUT | Edge cases and common mistakes

- Using a bitmask when `n` exceeds about 20, producing a table that will not allocate.
- Presenting bitmask DP as making TSP polynomial. It remains exponential and NP-hard.
- Relaxing from an unreachable state, so a sentinel participates in arithmetic and overflows.
- Using `Integer.MAX_VALUE` as an additive sentinel instead of a half-range value.
- Forgetting that increasing numeric mask order is already a valid topological order, and writing unnecessary ordering logic.
- Assuming subset-of-subset enumeration is O(4^n); it is O(3^n) by the three-choices argument.
- Memoizing tight states in digit DP, which is unnecessary and unsound across different bounds.
- Omitting the `started` flag in digit DP and mishandling leading zeros.
- Updating stock-state variables in place rather than from a snapshot, permitting an illegal same-day transaction.
- Treating each stock variant as a new problem instead of one more bounded dimension.
- Using `1 << k` with `k >= 31` on `int`; shift to `long` or the result is undefined for the sign bit.

TRY IT | Interview questions and model answers

When would you use a bitmask for DP state?

When the state must record *which* elements are used rather than how many, when order does not matter to the state, and when `n` is small enough that `2^n` is enumerable - roughly 20. A constraint of `n <= 20` in a prompt is usually the setter telling you this is the intended approach.

Does bitmask DP make TSP tractable?

It reduces brute force from `O(n!)` to `O(2^n * n^2)` via Held-Karp, which is a real improvement and makes `n` around 20 feasible. It does not make it polynomial and TSP remains NP-hard. For larger instances you move to heuristics or approximation, not a better exact DP.

Why is iterating masks in increasing numeric order correct?

Adding an element sets a previously clear bit, which strictly increases the integer value. So every predecessor of a mask is numerically smaller and therefore already computed. The natural integer order is a topological order of the state DAG, which is why no explicit ordering logic is needed.

What is the `tight` flag in digit DP?

It records whether the digits placed so far exactly match the prefix of the bound. When tight, the current digit is capped by the bound's digit; when not, all ten digits are available. It is what makes the state space finite and small. Only non-tight states should be memoized, since tight states are bound-specific and visited once.

Your stock DP with cooldown allows buying on the sell day. Why?

The state variables were updated in place, so the buy transition read the value already updated this iteration rather than yesterday's. Snapshot the previous values first, then compute all three from the snapshot.

How do you extend a two-state stock DP to at most k transactions?

Add a bounded dimension counting transactions used: `dp[day][used][holding]`. The transitions are unchanged in shape; selling increments `used`. This is the general pattern - a new rule usually becomes a small extra dimension rather than a different algorithm.

TRY IT | Exercises

1. **Foundation:** Write the four bit operations for testing, setting, clearing, and counting bits in a mask, and state what `Integer.bitCount` compiles to.
2. **Foundation:** For `n = 20` and `n = 25`, compute the table sizes for `dp[mask]` and `dp[mask][last]`. State which are feasible in 256 MB.
3. **Interview Core:** Implement minimum-cost assignment with bitmask DP. Then remove the unreachable-state guard and construct an input where the answer becomes negative.
4. **Interview Core:** Implement Held-Karp TSP. Report the wall time at `n = 12`, `16`, and `20`, and state where it stops being practical.
5. **Interview Core:** Prove that summing subset-of-subset enumeration over all masks is O(3^n), using the three-choices argument.
6. **Interview Core:** Implement digit DP counting integers in `[1, N]` whose digit sum is divisible by 7. Include the `tight` and `started` flags.
7. **Interview Core:** Memoize tight states in your digit DP and construct two different bounds that share a cache to show the wrong answer.
8. **SDE-2 Follow-up:** Implement all five stock variants from the table as one parameterized solution, and state which dimension each rule added.
9. **SDE-2 Follow-up:** Take the cooldown solution, update the variables in place rather than from a snapshot, and find the input where the profit becomes impossible.
10. **Challenge:** Given `n <= 18` items with weights and a target, count the number of distinct subsets summing to the target - first with bitmask enumeration, then with classic subset-sum DP. Compare complexities and say when each wins.

RECAP | Chapter summary

These three shapes cover the DP problems whose state is not an index. Bitmask DP encodes a subset in an integer and is gated hard by `n` around 20; its natural numeric iteration order is already a topological order, and its most common bug is relaxing from an unreachable sentinel that then overflows. It improves TSP from factorial to `O(2^n * n^2)` without making it polynomial, and saying so precisely matters more than the implementation. Digit DP walks a numeral left to right with a `tight` flag capping the current digit and a `started` flag handling leading zeros; only non-tight states may be memoized. Bounded-state DP adds a small counter - transactions, cooldown, budget - to an ordinary state, and the lesson from the stock family is that a new rule almost always becomes one more small dimension rather than a different algorithm, provided the transitions read a snapshot of the previous step rather than values already updated this one.

TRY IT | Revision checklist

- [ ] I can state the three conditions that justify a bitmask state.
- [ ] I know the practical `n` ceiling and can compute the table size.
- [ ] I can explain why increasing numeric mask order is a valid evaluation order.
- [ ] I guard against relaxing from unreachable states and use half-range sentinels.
- [ ] I can state Held-Karp's complexity and that TSP remains NP-hard.
- [ ] I can justify the O(3^n) bound on subset-of-subset enumeration.
- [ ] I can explain the `tight` flag and why only non-tight states are memoized.
- [ ] I know why digit DP needs a `started` flag.
- [ ] I can extend a stock DP to k transactions, cooldown, and fees by adding dimensions.
- [ ] I snapshot previous state before updating multi-variable transitions.

SDE-2 CORE CHAPTER

Chapter 8 - Dynamic Programming Practice Lab

1. Draw repeated states in naive Fibonacci recursion.
2. State House Robber using a half-open prefix.
3. Compare memoization and tabulation for sparse reachable states.
4. Debug a memo array whose zero value means both "not computed" and a valid answer.
5. Debug 0/1 knapsack compressed with upward capacity iteration.
6. Count grid paths with obstacles and define numeric overflow policy.
7. Return one longest increasing subsequence, not only its length.
8. Distinguish coin-change combinations from permutations by loop order.
9. Reconstruct an edit script from edit-distance state.
10. Derive stock-trading state as day and holding status.
11. Define interval DP for matrix-chain multiplication or burst balloons.
12. Define tree DP for maximum nonadjacent node sum.
13. Explain pseudo-polynomial complexity.
14. Decide when space compression is worth losing reconstruction data.
15. Compare DP, greedy, and shortest-path formulations of the same state graph.
16. **Interview Core:** Solve word break as prefix feasibility, return one valid segmentation, and discuss substring-allocation cost in Java.

State and reconstruction lab

17 **Foundation:** Write state, transition, base, order, and answer for minimum coins before coding.

18 **Interview Core:** Implement memoized and tabulated minimum coins with separate unknown and unreachable states; compare outputs.

19 **Interview Core:** Implement 0/1 knapsack with deterministic reconstruction, then explain why upward one-row capacity iteration changes the problem.

20 **Interview Core:** Implement edit distance in two rows and state whether units are UTF-16 code units, code points, or graphemes.

21 **Interview Core:** Derive a four-state at-most-two-transaction stock DP and reject invalid negative prices.

22 **SDE-2 Follow-up:** Compare knapsack DP with exhaustive subset enumeration and describe what independent bugs the oracle can and cannot catch.

SDE-2 CORE CHAPTER

Chapter 9 - Dynamic Programming Practice Lab Solutions

1 `fib(n-2)` appears directly and again inside `fib(n-1)`; the recurrence tree repeats many argument values.

2 `dp[i]` is the maximum amount from houses `[0,i)`. Transition is max of skipping house `i-1` and taking it plus `dp[i-2]`.

3 Memoization can avoid unreachable states but retains recursion overhead/depth. Tabulation has explicit order and good locality but may evaluate every table cell.

4 Use a separate visited array, nullable wrappers, or a sentinel outside the valid answer range. Do not confuse a legitimate zero with missing state.

5 Iterate capacity downward so each item reads only states from the previous conceptual item layer. Upward iteration permits reuse.

6 `dp[row][column]` counts paths to that cell; obstacles contribute zero, and other cells add top plus left. Use long, exact arithmetic, modulo, or big integers according to the contract.

7 O(n squared) DP stores predecessor index when a better length ends at i, then follows predecessors from the best endpoint. O(n log n) reconstruction tracks tail indexes and predecessors.

8 Coins outer and amounts upward count combinations without order. Amount outer and coins inner count ordered sequences under the corresponding recurrence.

9 Walk backward from `(m,n)`: a diagonal match emits nothing, diagonal mismatch emits replace, upward emits delete, and left emits insert. Tie-breaking defines which optimal script is returned.

10 `dp[day][holding]` is best profit from a day under current holding status. Transitions compare rest with buy/sell, extended for transaction count, fee, or cooldown.

11 State over interval `[left,right]`; try every final split/burst and combine independent subinterval answers plus boundary cost. Evaluate by increasing interval length.

12 Each node returns two values: best when node is taken and when skipped. Taking forces children skipped; skipping allows the better child state independently.

13 O(n * amount) is polynomial in numeric amount but exponential in the number of bits required to encode a potentially huge amount.

14 Compress only after dependency analysis. Keep the full table or parent choices when an actual solution path is required and memory permits.

15 DP evaluates a DAG of states, greedy commits using a proof, and shortest path optimizes edge-cost accumulation. Equivalent state graphs can reveal alternative algorithms, but edge weights and acyclicity determine validity.

16 Let `reachable[end]` mean prefix `[0,end)` can be segmented, with `reachable[0]=true`. From every reachable start, try dictionary words or endings up to the maximum word length; record the predecessor start on the first successful transition to each end. Reconstruct backward from n and reverse. The abstract transition count can be O(nL), but Java substring creation and hashing add character/allocation cost; a trie or region comparison can avoid temporary strings.

State and reconstruction solutions

17 `dp[a]` is minimum coins for exactly amount `a`, with `dp[0]=0`. For each amount, try positive coins whose predecessor is reachable and take `1+dp[a-coin]`. Evaluate amounts ascending. Return `dp[amount]` or -1. Complexity is `O(amount*coinCount)` time and `O(amount)` space.

18 Memo uses unknown marker distinct from cached unreachable -1, recursively trying smaller positive-coin amounts. Tabulation initializes an unreachable sentinel and fills ascending. Validate coins even when amount zero. Compare both over random small domains; use tabulation for amounts whose recursion depth is unsafe.

19 Use `best[i][c]` over first `i` items, exclude or include from row `i-1`, then walk backward: a changed value selects item `i-1` under exclusion-on-tie policy. Upward one-row iteration reads current-item updates and permits repeated use; downward preserves previous-item semantics.

20 `dp[i][j]` converts first `i` units to first `j`; equal final units take diagonal, otherwise one plus replace/delete/insert minimum. Keep previous/current rows and put shorter input on columns. Java `charAt` defines UTF-16 code-unit distance, not necessarily code-point or grapheme distance.

21 Track best balance after first buy, first sell, second buy, second sell. For each nonnegative price, update in transaction order with max of retaining state or performing its action. Initial sell states are zero; buy states begin negative infinity. Return second sell, which includes zero/one/two transactions.

22 Enumerate every subset for small `n`, sum weight/value, and retain best feasible value; compare with DP. It independently tests transition/value correctness and boundary capacity but can share input-validation assumptions and does not by itself verify reconstructed item identities unless those are also checked.

SDE-2 CORE CHAPTER

Chapter 10 - Executable Dynamic-Programming State Checks

This dependency-free Java 21 companion validates memo/tabulation contracts, bounded Fibonacci, reconstructing knapsack, edit distance, grid paths, transaction-state machines, compression, and independent oracles. A successful run prints PASS 20 dynamic-programming checks.

JAVA (continued 1/10)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Random;

public final class DynamicProgrammingInterviewChecks {
    record KnapsackResult(long maximumValue, List<Integer> chosenIndices) {}

    private DynamicProgrammingInterviewChecks() {}

    static int minimumCoins(int[] coins, int amount) {
        validateCoinInput(coins, amount);
        int unreachable = amount + 1;
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, unreachable);
        dp[0] = 0;
        for (int current = 1; current <= amount; current++) {
            for (int coin : coins) {
                if (coin <= current && dp[current - coin] != unreachable) {
                    dp[current] = Math.min(dp[current], dp[current - coin] + 1);
                }
            }
        }
        return dp[amount] == unreachable ? -1 : dp[amount];
    }

    static int minimumCoinsMemoized(int[] coins, int amount) {
        validateCoinInput(coins, amount);
        if (amount > 2_000) {
            throw new IllegalArgumentException("memoized teaching version limits call depth");
        }
        int[] memo = new int[amount + 1];
```

JAVA (continued 2/10)

```
        Arrays.fill(memo, -2);
        memo[0] = 0;
        return minimumCoinsMemoized(coins, amount, memo);
    }

    private static int minimumCoinsMemoized(int[] coins, int amount, int[] memo) {
        if (memo[amount] != -2) {
            return memo[amount];
        }
        int best = Integer.MAX_VALUE;
        for (int coin : coins) {
            if (coin <= amount) {
                int previous = minimumCoinsMemoized(coins, amount - coin, memo);
                if (previous >= 0) {
                    best = Math.min(best, previous + 1);
                }
            }
        }
        memo[amount] = best == Integer.MAX_VALUE ? -1 : best;
        return memo[amount];
    }

    private static void validateCoinInput(int[] coins, int amount) {
        if (amount < 0 || amount > 1_000_000) {
            throw new IllegalArgumentException("amount must be in [0,1000000]");
        }
        for (int coin : coins) {
            if (coin <= 0) {
                throw new IllegalArgumentException("positive coins required");
            }
        }
    }
```

JAVA (continued 3/10)

```
    static int lcsLength(String first, String second) {
        int[][] dp = new int[first.length() + 1][second.length() + 1];
        for (int i = 1; i <= first.length(); i++) {
            for (int j = 1; j <= second.length(); j++) {
                dp[i][j] = first.charAt(i - 1) == second.charAt(j - 1)
                        ? dp[i - 1][j - 1] + 1
                        : Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[first.length()][second.length()];
    }

    static boolean canPartition(int[] values) {
        long total = 0L;
        for (int value : values) {
            if (value < 0) throw new IllegalArgumentException("nonnegative values required");
            total += value;
        }
        if ((total & 1L) != 0L || total / 2L > Integer.MAX_VALUE) return false;
        int target = (int) (total / 2L);
        boolean[] possible = new boolean[target + 1];
        possible[0] = true;
        for (int value : values) {
            for (int sum = target; sum >= value; sum--) {
                possible[sum] |= possible[sum - value];
            }
        }
        return possible[target];
    }

    static long fibonacciMemoized(int n) {
        if (n < 0 || n > 92) {
            throw new IllegalArgumentException("n must be in [0,92]");
```

JAVA (continued 4/10)

```
        }
        long[] memo = new long[n + 1];
        Arrays.fill(memo, -1L);
        return fibonacciMemoized(n, memo);
    }

    private static long fibonacciMemoized(int n, long[] memo) {
        if (n < 2) {
            return n;
        }
        if (memo[n] != -1L) {
            return memo[n];
        }
        memo[n] = Math.addExact(
                fibonacciMemoized(n - 1, memo), fibonacciMemoized(n - 2, memo));
        return memo[n];
    }

    static long fibonacciConstantSpace(int n) {
        if (n < 0 || n > 92) {
            throw new IllegalArgumentException("n must be in [0,92]");
        }
        if (n == 0) {
            return 0;
        }
        long previous = 0;
        long current = 1;
        for (int index = 2; index <= n; index++) {
            long next = Math.addExact(previous, current);
            previous = current;
            current = next;
        }
        return current;
```

JAVA (continued 5/10)

```
    }

    static KnapsackResult knapsack01(
            int[] weights, int[] values, int capacity) {
        if (weights.length != values.length || capacity < 0 || capacity > 100_000) {
            throw new IllegalArgumentException("invalid knapsack dimensions");
        }
        long cells = (weights.length + 1L) * (capacity + 1L);
        if (cells > 10_000_000L) {
            throw new IllegalArgumentException("knapsack table exceeds teaching memory budget");
        }
        for (int item = 0; item < weights.length; item++) {
            if (weights[item] <= 0 || values[item] < 0) {
                throw new IllegalArgumentException(
                        "positive weights and nonnegative values required");
            }
        }
        long[][] best = new long[weights.length + 1][capacity + 1];
        for (int item = 1; item <= weights.length; item++) {
            int weight = weights[item - 1];
            int value = values[item - 1];
            for (int room = 0; room <= capacity; room++) {
                best[item][room] = best[item - 1][room];
                if (weight <= room) {
                    best[item][room] = Math.max(best[item][room],
                            best[item - 1][room - weight] + value);
                }
            }
        }
        List<Integer> chosen = new ArrayList<>();
        int room = capacity;
        for (int item = weights.length; item > 0; item--) {
            if (best[item][room] != best[item - 1][room]) {
```

JAVA (continued 6/10)

```
                chosen.add(item - 1);
                room -= weights[item - 1];
            }
        }
        Collections.reverse(chosen);
        return new KnapsackResult(best[weights.length][capacity], List.copyOf(chosen));
    }

    static int editDistance(String first, String second) {
        if (second.length() > first.length()) {
            return editDistance(second, first);
        }
        int[] previous = new int[second.length() + 1];
        for (int column = 0; column <= second.length(); column++) {
            previous[column] = column;
        }
        for (int row = 1; row <= first.length(); row++) {
            int[] current = new int[second.length() + 1];
            current[0] = row;
            for (int column = 1; column <= second.length(); column++) {
                if (first.charAt(row - 1) == second.charAt(column - 1)) {
                    current[column] = previous[column - 1];
                } else {
                    current[column] = 1 + Math.min(previous[column - 1],
                            Math.min(previous[column], current[column - 1]));
                }
            }
            previous = current;
        }
        return previous[second.length()];
    }

    static long minimumPathSum(int[][] grid) {
```

JAVA (continued 7/10)

```
        if (grid.length == 0 || grid[0].length == 0) {
            throw new IllegalArgumentException("nonempty grid required");
        }
        int columns = grid[0].length;
        long[] best = new long[columns];
        Arrays.fill(best, Long.MAX_VALUE);
        best[0] = 0;
        for (int[] row : grid) {
            if (row.length != columns) {
                throw new IllegalArgumentException("rectangular grid required");
            }
            for (int column = 0; column < columns; column++) {
                long fromAbove = best[column];
                long fromLeft = column == 0 ? Long.MAX_VALUE : best[column - 1];
                long previous = Math.min(fromAbove, fromLeft);
                best[column] = Math.addExact(previous, row[column]);
            }
        }
        return best[columns - 1];
    }

    static long maximumProfitAtMostTwoTransactions(int[] prices) {
        long buyFirst = Long.MIN_VALUE / 4;
        long sellFirst = 0;
        long buySecond = Long.MIN_VALUE / 4;
        long sellSecond = 0;
        for (int price : prices) {
            if (price < 0) {
                throw new IllegalArgumentException("prices cannot be negative");
            }
            buyFirst = Math.max(buyFirst, -(long) price);
            sellFirst = Math.max(sellFirst, buyFirst + price);
            buySecond = Math.max(buySecond, sellFirst - price);
```

JAVA (continued 8/10)

```
            sellSecond = Math.max(sellSecond, buySecond + price);
        }
        return sellSecond;
    }

    private static boolean coinMethodsAgree() {
        Random random = new Random(71L);
        for (int trial = 0; trial < 2_000; trial++) {
            int[] coins = new int[1 + random.nextInt(5)];
            for (int index = 0; index < coins.length; index++) {
                coins[index] = 1 + random.nextInt(10);
            }
            int amount = random.nextInt(80);
            if (minimumCoins(coins, amount) != minimumCoinsMemoized(coins, amount)) {
                return false;
            }
        }
        return true;
    }

    private static boolean knapsackMatchesSubsetOracle() {
        Random random = new Random(73L);
        for (int trial = 0; trial < 1_000; trial++) {
            int count = random.nextInt(12);
            int[] weights = new int[count];
            int[] values = new int[count];
            for (int item = 0; item < count; item++) {
                weights[item] = 1 + random.nextInt(8);
                values[item] = random.nextInt(20);
            }
            int capacity = random.nextInt(25);
            long expected = 0;
            for (int mask = 0; mask < 1 << count; mask++) {
```

JAVA (continued 9/10)

```
                int weight = 0;
                long value = 0;
                for (int item = 0; item < count; item++) {
                    if ((mask & 1 << item) != 0) {
                        weight += weights[item];
                        value += values[item];
                    }
                }
                if (weight <= capacity) {
                    expected = Math.max(expected, value);
                }
            }
            if (knapsack01(weights, values, capacity).maximumValue() != expected) {
                return false;
            }
        }
        return true;
    }

    private static void expectFailure(Runnable action) {
        try {
            action.run();
        } catch (IllegalArgumentException expected) {
            return;
        }
        throw new AssertionError("expected IllegalArgumentException");
    }

    private static void check(boolean condition, String message) {
        if (!condition) throw new AssertionError(message);
    }
```

JAVA (continued 10/10)

```
    public static void main(String[] args) {
        check(minimumCoins(new int[] {1, 2, 5}, 11) == 3, "minimum coins");
        check(minimumCoins(new int[] {2}, 3) == -1, "unreachable");
        check(lcsLength("abcde", "ace") == 3, "LCS");
        check(canPartition(new int[] {1, 5, 11, 5}), "partition true");
        check(!canPartition(new int[] {1, 2, 3, 5}), "partition false");
        check(minimumCoinsMemoized(new int[] {1, 2, 5}, 11) == 3,
                "memoized minimum coins");
        expectFailure(() -> minimumCoins(new int[] {0, 1}, 0));
        check(fibonacciMemoized(50) == 12_586_269_025L, "memoized Fibonacci");
        check(fibonacciConstantSpace(92) == 7_540_113_804_746_346_429L,
                "space-optimized Fibonacci boundary");
        expectFailure(() -> fibonacciMemoized(93));

        KnapsackResult knapsack = knapsack01(
                new int[] {2, 3, 4, 5}, new int[] {3, 4, 5, 8}, 8);
        check(knapsack.maximumValue() == 12L, "knapsack value");
        check(knapsack.chosenIndices().equals(List.of(1, 3)), "knapsack reconstruction");
        check(editDistance("horse", "ros") == 3, "edit distance");
        check(editDistance("", "abc") == 3, "edit empty boundary");
        check(minimumPathSum(new int[][] {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}}) == 7L,
                "minimum path sum");
        expectFailure(() -> minimumPathSum(new int[][] {{1}, {2, 3}}));
        check(maximumProfitAtMostTwoTransactions(new int[] {3, 3, 5, 0, 0, 3, 1, 4}) == 6L,
                "two stock transactions");
        check(maximumProfitAtMostTwoTransactions(new int[0]) == 0L,
                "empty price series");
        check(coinMethodsAgree(), "memo/tabulation differential test");
        check(knapsackMatchesSubsetOracle(), "knapsack subset oracle");
        System.out.println("PASS 20 dynamic-programming checks");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: Fibonacci, stairs, and one-dimensional choices
- Interview Core: grids, knapsack, subsequences, and string DP
- SDE-2 Follow-up: reconstruct decisions and control memory
- Challenge: interval, tree, or bitmask state

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 16 - Greedy Algorithms: Recognition, Proofs, and Scheduling

Series index

Return to the complete series index

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design
System Design	**SD 03**	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 17 of 17 - Study Step DSA-17. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.